



Timely Real-Time Intervention and Guidance System for Supporting Students in Labs

Abubakar Othman (F221611)
MSci Computer Science and Mathematics
Department of Computer Science
Supervisor: Dr Firat Batmaz
Module Code: 25COD290

Final Deliverable 2026

Abstract

Contents

1	Introduction	6
1.1	Problem	6
1.2	Proposed Solution	6
1.3	Aims and Objectives	6
1.4	Risks and Mitigation	7
1.5	Project Approach	7
2	Background and Requirements	9
2.1	Learning Analytics and Dashboards	9
2.1.1	Learning Analytics in Higher Education	9
2.1.2	Dashboards for Learning Analytics	9
2.1.3	Real-Time Data Analytics and Visualisation	11
2.2	Modelling Student Behaviour	12
2.2.1	Modelling Student Struggle	12
2.2.2	Knowledge Tracing and Bayesian Student Models	12
2.2.3	Modelling Task Difficulty	13
2.3	Data-Driven Personalisation	14
2.3.1	Collaborative Filtering	14
2.3.2	Text Mining and Mistake Pattern Recovery	14
2.4	Retrieval-Augmented Feedback	15
2.4.1	Retrieval-Augmented Generation	15
2.4.2	Dense Retrieval and Approximate Nearest Neighbour Search	15
2.5	Existing Systems	16
2.5.1	Learning Management System	16
2.5.2	Instructor Facing Dashboards	16
2.5.3	Early Warning and At-Risk Student Systems	18
2.6	Research Gaps	19
2.7	Requirements Specification	19
2.7.1	Functional Requirements	19
2.7.2	Non-Functional Requirements	20
2.7.3	Requirements Prioritisation (MoSCoW)	21
3	Design and Modelling	22
3.1	System Architecture	22

3.2	Data Endpoint	23
3.3	Baseline Analytics Design	24
3.3.1	Student Struggle Model	24
3.3.2	Temporal Smoothing	27
3.3.3	Question Difficulty Model	28
3.3.4	Alternative Modelling Approach: Collaborative Filtering	29
3.3.5	Mistake Clustering	31
3.4	Advanced Model Design	32
3.4.1	Measurement Confidence	32
3.4.2	Item Response Theory	32
3.4.3	Bayesian Knowledge Tracing	34
3.4.4	Improved Struggle Model	35
3.5	Retrieval-Augmented Generation Feedback Design	36
3.6	Interaction and UI Designs	38
3.6.1	Design Principles	38
3.6.2	Instructor Views	38
3.6.3	Lab Assistant View	40
3.6.4	Visual Encoding of Metrics	41
4	Implementation	43
4.1	Implementation Overview	43
4.2	Technology Stack	44
4.3	System Structure	44
4.3.1	V1 - Instructor Process (Streamlit)	45
4.3.2	V1 - Assistant Process (Streamlit, mobile)	45
4.3.3	V2 - React + FastAPI	46
4.3.4	Shared Runtime State	48
4.3.5	Deferred-Actions Pattern	49
4.4	Data Pipeline	50
4.5	Session Management	53
4.6	Analytics Implementation	56
4.6.1	Incorrectness Scoring	56
4.6.2	Baseline Student Struggle Model	56
4.6.3	Baseline Question Difficulty Model	56
4.6.4	Collaborative Filtering	56
4.6.5	Mistake Clustering	56

4.7	Advanced Model Implementation	56
4.7.1	Measurement Confidence	56
4.7.2	Item Response Theory (IRT) Difficulty	56
4.7.3	Bayesian Knowledge Tracing (BKT) Mastery	56
4.7.4	Improved Struggle Model	56
4.8	RAG Suggested Feedback	56
4.8.1	Hybrid Two-Layer Architecture	56
4.8.2	Embedding Model (all-MiniLM-L6-v2)	56
4.8.3	Generation and Prompting	56
4.8.4	Suggestion Caching	56
4.8.5	Graceful Degradation and Privacy	56
4.9	Instructor System Views	56
4.9.1	In-Class View	57
4.9.2	Student Detail View	57
4.9.3	Question Detail View	57
4.9.4	Data Analysis View	57
4.9.5	Model Comparison View	57
4.9.6	Settings View	57
4.9.7	Previous Sessions View	57
4.10	Lab Assistant System	57
4.10.1	Session Join Flow	57
4.10.2	Waiting and Assignment States	57
4.10.3	Live Assistant Allocation and Mark-Helped	58
4.11	Problems Encountered and Solutions	58
5	Evaluation	59
5.1	Evaluation Design	59
5.1.1	Scope of Evaluation	59
5.1.2	Evidence Sources	59
5.1.3	Retrospective Evaluation Protocol	59
5.1.4	Limitations of Evaluation	59
5.2	Functional Testing	59
5.2.1	Data Ingestion and Session Handling	59
5.2.2	Analytics and Model Behaviour	59
5.2.3	Instructor Dashboard Features	59
5.2.4	Lab Assistant Workflow	59

5.3	Non-Functional Testing	59
5.3.1	Performance and Refresh Behaviour	59
5.3.2	Usability and Interpretability	59
5.3.3	Robustness and Failure Handling	59
5.4	Results	59
5.4.1	Baseline System Results	59
5.4.2	Advanced Model Comparison	59
5.4.3	Observed Strengths and Weaknesses	59
5.5	Discussion	59
5.5.1	Model Disagreement	59
5.5.2	Limitations	59
6	Conclusion	60
6.1	Summary of the Project	60
6.2	Key Findings Contributions	60
6.3	Future Work	60
A	Code Snippets	61
B	User Interface Screenshots	62
C	Detailed Test Results	63
D	Themes and References	64
E	Formulae	65
F	Formulae Derivation	66

1 Introduction

1.1 Problem

Computer Labs are fast paced environments where many students are working simultaneously on different questions at their own pace. Some students complete questions slower than others, and some may have prior knowledge. In practice, it is difficult to provide timely, high-quality support to students, monitor their progress, and prioritise those who are struggling most so they do not fall behind. We aim to address these issues in this paper.

Our current computer labs are highly dependent on staff surveying the room or counting on students asking for help. The current approach to computer labs has a range of limitations. Students who hesitate to ask for help in a lab environment may not receive the support they need. A student can be struggling significantly, but we may have no indicator of this struggle. Additionally, knowing what students require more help is almost impossible with the current approach, even with staff surveying the lab it is hard to quickly judge who needs immediate help. Furthermore, there is no structure to the allocation of lab assistants, without any indicator of who is struggling and allocating students smartly we do not provide help in the most efficient way possible.

These limitations become more severe the more class size increases. With more students, it becomes harder to identify struggling students and prioritise them for help.

The central problem addressed in this project is therefore how to provide timely, high-quality feedback to students in computer labs, using the data generated from student work. The goal is not to replace human teaching but to enhance it.

1.2 Proposed Solution

In this project, we have addressed these issues and limitations with a real-time learning analytics dashboard we developed for use in computer lab sessions. The system supports lab teachers and assistants by transforming live student data into interpretable metrics we can use quickly during labs.

The dashboard integrates data from an existing endpoint and processes it into a set of practical metrics. These include measures derived from attempt number, time spent and average numbers of attempts to complete a question. From these signals we compute indicators of student struggle and question difficulty, allowing staff to identify which students are struggling and which questions are causing the most difficulty.

The system aims to assist rather than replace human judgement. The dashboard does not by default make its own decisions. Instead it provides a clearer operational view of what is happening in the lab so instructors can allocate resources more effectively. The system also allows for lab instructors to allocate lab assistants within the session, allowing them to be allocated through the system rather than informally.

The project has smart device integration, sending who to help to lab assistant's phones with recommendations on how to help students.

1.3 Aims and Objectives

The primary aim of this project was to design and implement a system that enables lab teachers and lab assistants to make more well-informed decisions during computer lab sessions.

The secondary aim is to extend our system to smart devices. Whilst the core system is a desktop-based dashboard, we can improve usability by sending notifications or messages to smart devices to enable easier communication with assistants.

The aims will be addressed through the following objectives:

1. **Identify relevant literature and existing systems** related to learning analytics, real-time data visualisation, and AI in education, to identify existing limitations in current approaches to relevant areas
2. **Formalise system requirements** by defining functional and non-functional requirements for the real-time computer lab support system
3. **Design Data Model** that extracts relevant data provided into meaningful metrics suitable for real-time analysis
4. **Design system architecture** to support live data ingestion, prioritisation and presentation while remaining extensible to smart devices
5. **Develop a struggle and difficulty model** so that we can take the relevant values to produce metrics which will be used in showing the relevant information
6. **Implement live data analytics dashboard** for real-time data visualisation, prioritisation and intuitive interactivity
7. **Evaluate approach** through functional and non-functional aspects to determine the feasibility and usefulness in a computer lab session

Together, these objectives guided the project toward becoming a full-fledged EdTech application that aimed to improve learning outcomes by enabling more efficient and targeted support.

1.4 Risks and Mitigation

The project involved developing a real-time data analytics system for use during computer lab sessions. As such, potential technical and methodological risks were identified and addressed through deliberate design decisions and extensive testing.

1.5 Project Approach

The project followed an incremental and implementation-focused approach. Early work focused on understanding the educational and technical context through a literature review and analysing the dataset. This established the problem framing, clarified the type of analytics that would be most useful to the lab instructor during a computer lab and also addressed the limitations of current approaches.

The next stage focused on system design. This included defining the system architecture, understanding the structure of the data endpoint and deciding which data can aid in creating our student struggle and question difficulty metrics. The process also also considered how the system could be extended to smart devices, ensuring that the design was flexible enough to accommodate future extensions.

Implementation then progressed from core functionality to increasing the complexity of our analytics. The priority was to establish reliable data retrieval, parsing and processing. Once this was in place, we extended the dashboard to include live monitoring, drill down analysis, model comparison, historical sessions and assistant coordination. More advanced analytical components were implemented such as collaborative filtering, mistake clustering and alternative modelling approaches which use Item Response Theory and Bayesian Knowledge Tracing.

The incremental approach was appropriate for this project as it reduced risk and made it possible to validate each layer of the system before moving on to the next step. This was approach allowed us to turn a functioning pipeline into a more complete analytics platform.

Risk Description	Potential Impact	Mitigation Measures
Latency in real-time data processing	Delays in updating the dashboard can reduce the usefulness and impact of timeliness of support	Ensure there are lightweight analytics and incremental updates
Unstable struggle and difficulty estimates due to insufficient data	Irregular interactions may lead to incorrectly identifying struggling students	Establish a baseline for suitable data or use an LLM to identify whether data is suitable
Overly complicated dashboard	Overcomplicating the dashboard may be more distracting and be unintuitive	Seek constant feedback on the UI so that we can implement it correctly and focus on main metrics
Unable to implement smart glasses functionality	Due to technical constraints, we may not be able to fully integrate the dashboard to smart glasses	Explore smartwatch solutions or simplify what goes into the smart glasses
Limited targeted towards different modules	A model that works well on one model may not work well on another	Design models over generic interaction patterns rather than specific module indicators

Table 1: Identified Risks and Mitigation Strategies

Overall, the project aligned with the practical aim of the dissertation, which was to produce a working system that improves awareness and prioritisation in live computer labs.

2 Background and Requirements

2.1 Learning Analytics and Dashboards

2.1.1 Learning Analytics in Higher Education

In 2012, the use of data analytics was shown to improve student success by providing real-time feedback, demonstrating the impact of data informed intervention in higher education [?]. Learning analytics is the measurement, collection, analysis, and reporting of data about learners and their contexts in order to understand and optimise learning and the environment in which it occurs [?, ?, ?, ?]. The growth of online education since the 1990s has contributed significantly to the advancement of learning analytics, as student data can now be captured and analysed enabling educators to make more data-driven decisions [?, ?, ?]. Learning analytics allows educators to make data-driven decisions.

Learning analytics leverages data sources; many types of data sources have been explored and used to help educators. Relevant data sources include Massive Open Online Courses (MOOCs), clickstream data (records of user interactions, such as clicks and time spent), and course content [?]. There are also data sources that, in the context of our project, we can get away without using, such as demographic data and gaze patterns. Whilst 35% of the studies used more than one data source, the majority of studies still used only one data source. It is argued that the use of multiple data types will contribute more effectively to learning outcomes [?].

The outputs of learning analytics have proven to be valuable for pedagogical decision-making. Use includes personalisation and feedback, which can tailor recommendations to students' behavioural profiles to promote diverse learning [?]. Another use case is the VITAL project, which confirmed that flipped classroom models were effective, as students accessed materials before attending face-to-face sessions [?]. However, several studies note challenges and limitations in learning analytics. These include how analytics leverage observable behaviours such as clicks and time online rather than cognitive processes [?]. Learning analytics also raises ethical concerns, such as privacy, when moving solutions out of labs [?, ?]. Many restrictions must be determined, as protecting students' identities and data is a priority, which we shouldn't lose sight of when trying to improve learning outcomes.

With the current utilisation of learning analytics, there is yet to be an approach that leverages smart devices in computer labs, in particular, smart watches and smart glasses. By leveraging smart devices, we can significantly increase the efficiency of lab assistance. From a centralised point, we can allow the lab teacher to send instructions to lab assistants based on these analytics, such as which questions are hardest and which students need the most help, thereby improving learning outcomes [?, ?, ?].

2.1.2 Dashboards for Learning Analytics

Learning analytics have been leveraged to use dashboards as valuable tools for both educators and learners, such as making educators aware of subtle interactions in their courses and allowing learners to gain insight into their learning actions and the effects they may have [?, ?]. Because of capabilities such as these, dashboards are the primary representation of learning analytics, as they can help improve learning outcomes and make the most of the data generated by learning analytics [?, ?].

SAM was an early dashboard developed on data from learning environments, with both teachers and learners as target user groups [?]. SAM allowed teachers to separate students who were doing well from those at risk based on time spent and resource-use metrics. Teachers gain insight into the time learners spent on tasks and can compare with their estimates, whilst resource use provides insights on how and how often students used documents and tools, it also allows discovery of resources external to the course [?]. Metrics could also be added, depending on data detail, to increase understanding of specific tasks [?]. EMODA is a more recent approach to the learning dashboard; it takes into account

emotion, which researchers have shown to play a key role in learning, regulation processes, and strong coupling to motivation and achievement [?]. EMODA enables teachers to monitor learners' emotions during online learning sessions by analysing audio, video, self-reports, and interaction traces [?].

Edsight is a web-based learning analytics dashboard developed with responsive web design that accommodates different device types and is accessible to educators [?]. When first logging in, teachers are presented with all the practical measure data they have collected thus far, the classrooms with the most data, and how many days have elapsed since they last logged data for analysis [?].



Figure 1: “Overview,” “Report,” and “Activity List” modes, displayed in desktop, tablet, and phone screens.

The report is the main dashboard in Edsight, where teachers can select filters such as date and classroom for display. Once a filter is applied, different results are displayed.

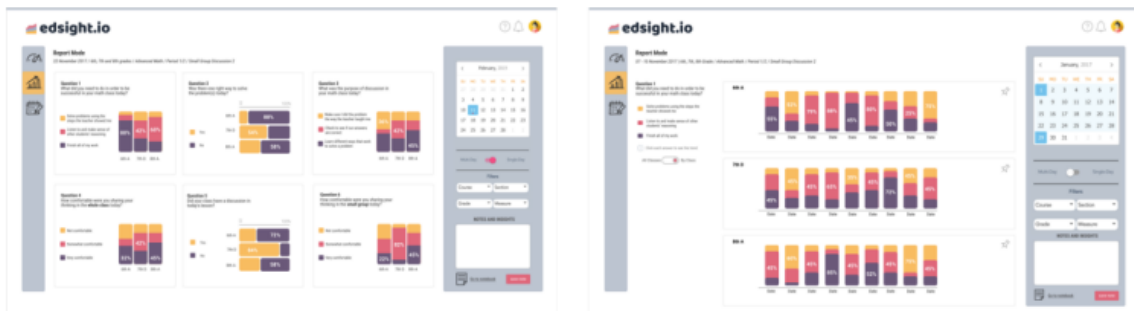


Figure 2: (L) A single-event report displayed in Edsight, with filters on the right side. (R) Multi-Day view, with the ability to compare patterns across classes and sections.

Most current approaches to the learning dashboard do not live in lab sessions, as there is more course-level monitoring than session-level monitoring [?, ?, ?]. This also has many benefits, helping students on the spot and allowing them to get the most out of their computer labs [?].

2.1.3 Real-Time Data Analytics and Visualisation

Real-time analytics refers to the practice of collecting and analysing data as it is generated, with minimal latency between data collection and analysis [?]. When we can process data as it comes in, we can gain insight and understand what is happening within a relevant context almost instantly. Such a practice is used in applications where data timeliness is critical, such as smart pricing and predictive maintenance [?]. Traditional data analysis, where data is captured and stored, and insights are pushed out of storage. In contrast, real-time data analysis allows data to be analysed in real time. Real-time data analytics have proven to be highly valuable, as evidenced by the success of Netflix, which has perfected the art of predicting its customers' next favourite show [?].

Real-time analytics has emerged as a pivotal tool across multiple sectors, including retail, revolutionising decision-making processes and operational strategies [?]. Retail giants such as Walmart and Amazon have leveraged real-time data analytics to personalise marketing campaigns, optimise pricing strategy, and efficiently manage inventory [?]. Technologies such as RFID, IoT, and advanced analytics platforms were used to gather real-time data, analyse customer behaviour, and make data-driven decisions [?]. A less utilised but more relevant application is in education, for example, a real-time machine learning application that detects learners' attention levels and participation using sensory data. The data allows teachers to recalibrate their teaching as necessary [?].

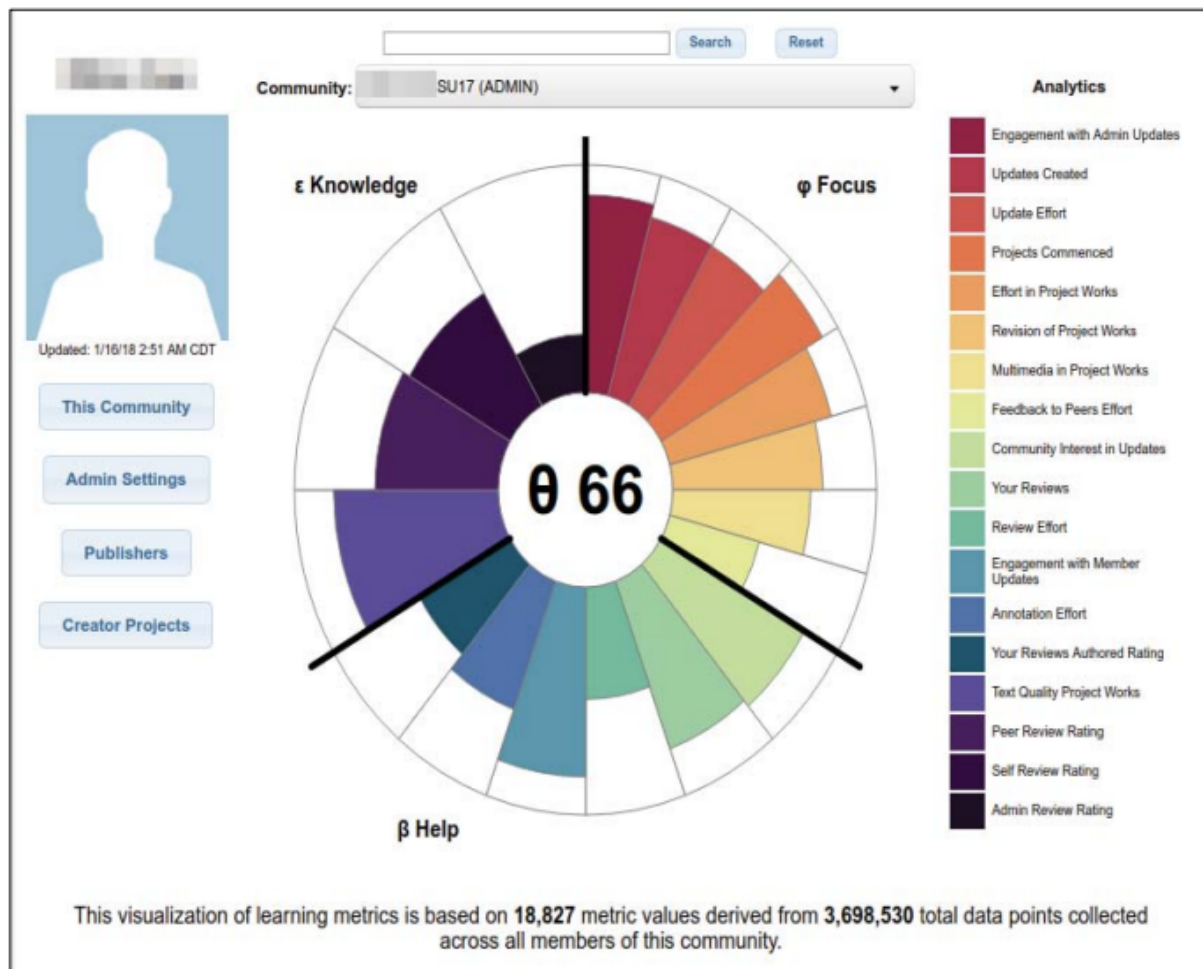


Figure 3: The Learning Analytics Interface for a dashboard which collects data based on student attention

Applying real-time analytics in computer labs will prove to be very beneficial for students and teachers alike. By collecting and processing data as it comes in, we will be able to update dashboards in real time and provide teachers with the most up-to-date information, enabling relevant, timely support. This would be better than a traditional data analysis approach, in which we review the data after the lab has finished, because our findings may not be relevant by then [?, ?].

2.2 Modelling Student Behaviour

2.2.1 Modelling Student Struggle

A key challenge in supporting students during lab sessions is identifying which students are struggling and may require timely intervention. Studies in education have approached struggle detection at different temporal scales, ranging from within-assignment detection to course-level. Dong et al. highlighted an approach that detects moments of struggle during a session based on time spent and progress relative to peers, achieving over 77% accuracy when validated against expert judgement on whether intervention was needed [?]. Struggle has been operationalised as failing the same unit test consecutively for more than four submission events, providing a clear threshold-based criterion [?].

Beyond threshold-based approaches, more sophisticated methods have been explored. Or examined novice programmers' testing behaviour as an early step toward defining coding struggle, identifying patterns such as repeatedly submitting the incorrect answer for more than four submission events, signalling that students have stopped making meaningful progress [?]. To move this even further, Estey et al. proposed a trajectory metric that measures changes in programming behaviour over time across a four-semester study of 514 students [?]. Their early predictor identified 81% of students who struggled throughout the course, but also flagged 43% who went on to succeed, highlighting the difficulty of distinguishing slight from serious struggle. Their trajectory metric reduced this false-positive rate to 11%, demonstrating the importance of tracking how performance changes rather than measuring the students' current level [?].

Another approach uses machine learning to build a graphical model of how students navigate through programming assignments. By capturing the full solution trajectory, Piech et al. showed that a student's progression path is predictive of later difficulty, motivating the inclusion of the student's trajectory to track changes in answer quality across submissions [?].

A complementary approach is knowledge tracing which models how a student's mastery of underlying skills progresses over time and it is examined in the next subsection.

Recent work on Large Language Models (LLMs) is also relevant as AI-generated feedback can provide more valuable insights than raw correctness alone. Koutchme et al. show that language models LLMs that are better at helping repair student programs also provide better explanations of coding issues, suggesting that the quality of LLM feedback can be studied empirically rather than assumed [?] More broadly, Pitts et al. survey LLM-based applications in programming education and argue that these systems are most effective when combined with human oversight rather than fully autonomous replacements [?]

Together, these works demonstrate different methods for evaluating struggle. These metrics are best captured through a combination of temporal, correctness-based and behavioural signals.

2.2.2 Knowledge Tracing and Bayesian Student Models

A related but distinct line of work is knowledge tracing, which models how a student's mastery of underlying skills progresses over time. Bayesian Knowledge Tracing (BKT) remains attractive because it is interpretable and computationally manageable. Khajah shows that BKT can gain a performance

boost by merging it with Item Response Theory (IRT) to model multidimensional problem difficulties and student abilities, without fitting student specific parameters, allowing the model to be generalisable to new students [?]. In programming education, Kim et al. propose a knowledge tracing approach that integrates students' questions and automatically extracted skill information, showing that textual interaction data can provide useful signals about understanding misconception patterns [?]. This is relevant to the current project, as it suggests that struggles during computer labs should not be treated solely as repeated failures but also as informed by temporal behaviour, how the student develops, and question-specific context.

Bayesian Knowledge Tracing was formalised by Corbett and Anderson in the context of an intelligent programming tutor, which treats the student's knowledge of each skill as either in a learned state or an unlearned state that can transition from unlearned to learned based on the student's interactions [?]. The model is governed by four interpretable parameters: an initial-learning probability, $p(L_0)$, an acquisition probability, $p(T)$ that a skill transitions from unlearned to learned at each practice opportunity, a guess probability, $p(G)$ for correct responses without mastery and a slip probability, $p(S)$ for incorrect responses despite mastery. The probability of a correct response on the next opportunity is then

$$p(C) = p(L) \cdot (1 - p(S)) + (1 - p(L)) \cdot p(G), \quad (1)$$

where $p(L)$ is the current estimate of mastery [?]. The original formulation assumes no forgetting, so the latent state cannot transition from learned back to unlearned.

Yudelson et al. extended BKT by individualising parameters across students and showed across KDD datasets that this yielded tangible improvements when predicting unseen student data [?]. They found that individualising the speed of learning $p(T)$ produced consistent gains in predictive accuracy, while individualising the prior $p(L_0)$ alone had no measurable effect, suggesting learning rate is the more important axis of student variation for BKT to exploit.

More recent work has explored neural alternatives to BKT such as deep learning approaches. Piech et al. proposed Deep Knowledge Tracing (DKT), with two different types of Recurrent Neural Networks (RNNs): a Long Short Term Memory (LSTM) based model and a vanilla RNN model with sigmoid unit. These models predict a student's future performance based on their past activity. This resulted in a 25% gain in the area under the curve (AUC) over the best previous result on a knowledge tracing benchmark as well as showing that knowledge tracing models do not need expert annotations [?]. However, Khajah et al. subsequently showed that BKT extended with forgetting, latent student abilities, and automatic skill discovery matches DKT performance on average across four different data sets, arguing that the gains of deep approaches stem from added flexibility rather than from any representation discovery [?]. For an instructor facing dashboard, where the interpretability of mastery estimates is essential, BKT remains a defensible choice.

2.2.3 Modelling Task Difficulty

Alongside students' struggles, it is important to identify which questions pose the greatest difficulty for the class as a whole so that teachers can address the problem in front of the whole class rather than just for the student. Dannath et al. examined task-level struggle detection methods in intelligent tutoring systems for programming, noting that prior work focused on dropout prevention rather than understanding the student's path towards the correct solution for a specific task [?]. The separation in modelling is important as students may appear to be struggling with a genuinely difficult question rather than with individual performance. Additionally, if most of the class is struggling with a problem, it will be faster to go through the question with the whole class.

Task difficulty can be inferred from indicators such as failure rate, average time spent, number of attempts and the amount of feedback used before reaching a correct answer [?, ?]. These measures are useful as they are directly observable and can be updated during a live session with low cost, but they do not separate question difficulty from variation in student ability.

Alternatively, we can model difficulty using Item Response Theory (IRT), which offers a more principled approach by accounting for how difficult a question actually is, rather than raw counts, as well as a student's latent ability [?, ?]. Baucks et al. uses IRT to analyse course difficulty variation and show how applying latent variable models can uncover meaning patterns in course rigor that is hidden when relying on simple metrics [?]. In online learning environments that allow multiple attempts, Pankiewicz argues that measuring task difficulty becomes more complex because task difficulty and feedback shape how it should be interpreted, motivating methods that go beyond raw metrics. [?] These works support the view that question difficulty should be modelled separately from student struggle and, where possible, that composite measures should be used rather than a single measure.

Overall, the literature suggests that task difficulty should be deduced from a combination of correctness, time, attempts, and progression-related information rather than from a single descriptive statistic [?, ?].

2.3 Data-Driven Personalisation

2.3.1 Collaborative Filtering

Beyond parametric approaches discussed in the previous subsection, more sophisticated data-driven methods have been explored in the educational recommender systems literature. Collaborative filtering is an alternative method used for personalising user experiences [?]. Unlike methods that rely on parameters, Collaborative filtering leverages the collective behaviour of users with similar histories to create preferences, on the principle that users with similar histories share similar needs. This method has been applied primarily in educational contexts to provide more efficient and accurate course suggestions than traditional methods. This can be done with different types of models, such as linear regression and factor decomposers. It is evident that integrating intelligent recommendation technology significantly enhances resource utilisation and the overall student learning experience

A user-based collaborative filtering method identifies a set of k nearest neighbours for a given student based on a similarity measure over their profiles [?]. The behaviour of the neighbours are aggregated to produce a prediction. The approach is shown to be effective in sparse datasets, where explicit ratings are unavailable and implicit signals such as time-on-task and submission frequency must serve as proxies for preference. A known limitation is that when a student has little interaction history, similarity cannot be computed reliably, leading to unstable early-session predictions.

In the context of lab struggle detection, collaborative filtering offers a strong alternative to our scoring approach. Rather than computing a fixed weighted score for each student, it can identify students whose interaction patterns closely resemble peers who need help, inferring that they too may need help. However applying collaborative filtering to a live computer lab comes with its constraints as students who may need help is limited and the start of the session will have very limited data. These constraints backs the use of a scoring approach, whilst the collaborative filtering approach is a strong alternative.

2.3.2 Text Mining and Mistake Pattern Recovery

Where student responses are free-text rather than numerical or categorical, identifying common mistake patterns requires text-mining techniques that combine information retrieval and unsupervised clustering. Salton et al. introduced the vector space model for information retrieval, which represents documents and queries as vectors in a vector space, allowing for the computation of similarity between them [?]. The standard weighting scheme is term frequency-inverse document frequency (TF-IDF) formalised in modern information retrieval as

$$\text{tf-idf}_{t,d} = \text{tf}_{t,d} \cdot \log \frac{N}{\text{df}_t}, \quad (2)$$

where $\text{tf}_{t,d}$ is the frequency of term t in document d , N is the total number of documents, and df_t is the number of documents containing term t [?].

Once each document is represented as a TF-IDF vector, similar documents can be partitioned into clusters that share characteristic content. MacQueen’s k -means algorithm partitions n points into k clusters to minimise within cluster variance [?]. Formally, this corresponds to choosing k centres C that minimise the sum of squared distances from each point to its closest centre,

$$\phi = \sum_{x \in X} \min_{c \in C} \|x - c\|^2, \quad (3)$$

where X is the set of data points [?]. Because this objective is non-convex and sensitive to initialisation, Arthur and Vassilvitskii’s k -means++ seeds initial centres by selecting each new centre with probability proportional to its squared distance from the nearest already-chosen centre, yielding provable approximation guarantees and faster convergence in practice [?].

Choosing the number of clusters k is itself a problem. Rousseeuw’s silhouette coefficient quantifies how well each point fits its assigned cluster relative to the next-nearest cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad (4)$$

where $a(i)$ is the mean intra-cluster distance for point i and $b(i)$ is the mean distance from i to all points in the nearest other cluster, with values close to +1 indicating tight well separated clusters [?]. Sweeping k and selecting the value that maximises the mean silhouette gives a principled auto-selection rule. Once clusters are formed, a large language model can produce natural-language cluster explanations [?].

2.4 Retrieval-Augmented Feedback

2.4.1 Retrieval-Augmented Generation

Lewis et al. introduced Retrieval-Augmented Generation (RAG) as a way to combine parametric language models with a non-parametric retriever over an external source, allowing the generator to condition its output on relevant evidence retrieved at query time [?]. Additionally, RAG models hallucinate less and generate factually more correct text than BART [?]. Lewis et al. showed that RAG outperformed parametric models on knowledge intensive tasks, including open-domain QA tasks [?].

In educational contexts, the case for retrieval is even stronger. Kasneci et al. survey the opportunities and challenges of large language models in education, noting that there is a difficulty in distinguishing between real knowledge and convincingly written but unverified model output due to the ability of LLMs to generate human-like text which can make it difficult for students to distinguish between real knowledge and unverified model output leading students to accept false or misleading information as true. To mitigate this risk, it is recommended we provide education on how information can be evaluated critically and teach students exploration, investigation, verification and corroboration strategies [?]. Retrieval-augmented generation can help mitigate this risk by grounding the model’s output in verifiable sources, allowing students to trace the information back to its origin and evaluate its credibility [?, ?].

2.4.2 Dense Retrieval and Approximate Nearest Neighbour Search

Before the retrieval step in RAG, we need a representation that captures semantic similarity between documents and queries. Reimers and Gurevych proposed Sentence-BERT (SBERT), which is a modification of pre-trained BERT that uses siamese and triplet network structures to derive semantically meaningful sentence embeddings that can be compared using cosine similarity [?]. In the paper they show that commonly used approaches to derive sentence embeddings from BERT such as CLS-token outputs or averaging the BERT output embeddings perform worse than averaged GloVe embeddings on standard semantic textual similarity benchmarks. Siamese fine-tuning on natural language inference data substantially

improves correlation on STS benchmarks and outperforms sentence-embedding methods such as InferSent and Universal Sentence Encoder [?].

Once documents are embedded, exact nearest neighbour search scales linearly with the number of documents which is unfeasible for large retrieval over large collections. To relax this, Approximate Nearest Neighbours Search (K-ANNS) was proposed, which relaxes the condition of the exact search by allowing a small number of errors in exchange for faster than linear time complexity [?]. Malkov and Yashunin proposed Hierarchical Navigable Small World (HNSW) graphs, which are a fully graph based ANN index that incrementally builds a multi-layer structure of proximity graphs for nested subsets of the stored elements [?]. As queries descend greedily from top to the zero layer, logarithmic complexity is achieved, with the evaluation showing that HNSW outperforms previous open source vector only approaches [?].

Together retrieval-augmented generation, dense sentence embeddings, and approximate nearest neighbour search build the standard pipeline for grounding LLM output.

2.5 Existing Systems

2.5.1 Learning Management System

A learning management system is a software application used for the administration, documentation, tracking, reporting, automation, and delivery of educational courses, training programs, materials or learning and development programs [?]. Learning management systems also constitute the largest share of the learning systems market; almost all higher education institutions have adopted them [?]. They were designed to identify training and learning gaps using analytical data and reporting. A notable learning management systems is Moodle which is the world's most popular learning management system [?].

A limitation of these systems are that the analytics are typically reviewed after activities, meaning some data may not be as useful after and students that were struggling in computer labs may have not been provided timely assistance. Another limitation of such systems is that they are primarily designed for course level and while this is useful for curriculum design and long-time planning, they are generally not optimised for in lab computer sessions where timely support is critical.

2.5.2 Instructor Facing Dashboards

Learning management systems often incorporate dashboards to track student or user progress, helping teachers better understand knowledge gaps [?]. Dashboards provide insight into student progress through visualisation and scores derived from data generated during teaching and learning activities. As a result, dashboards are at the centre of learning analytics, integrated with learning management systems [?].

Existing learning dashboards, such as Blocks: Progress Bar, report on student progress rather than student struggle at the live lab level [?]. Dashboards like these show user active time, assignment progress, learning progress, marks, and assignment submissions [?, ?, ?]. Visual components such as the progress bar aid in the understanding of student progress, removing all the complexity that is going on behind the scenes. Although these are useful, they don't directly present the user with more complex questions compared to others, and students struggle.

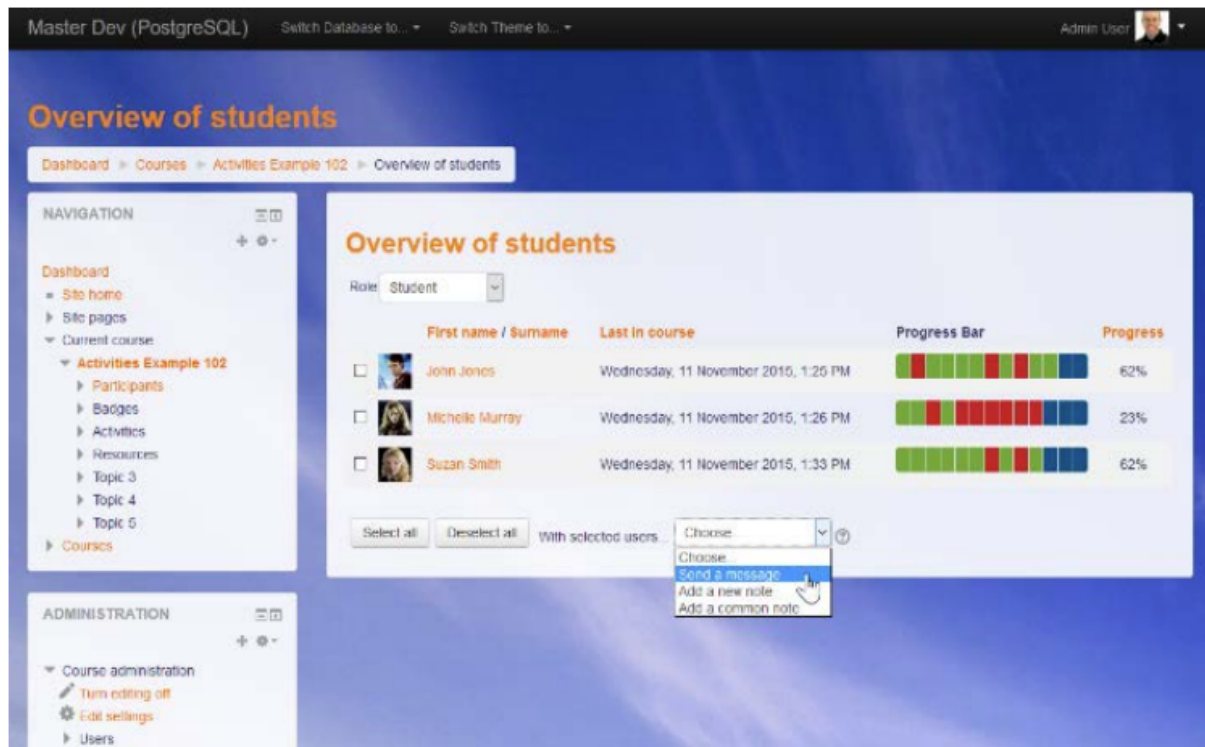


Figure 4: Piwik Analytics

Another implementation of a learning dashboard is the Multi-Modal (MM) teacher dashboard, the MM dashboard: 1) receives data from eye trackers, wristbands, web cameras, and the learning activity; 2) computes the student's relevant cognitive, affective, and physiological measurements; 3) synchronises and visualises the resulting measurements in a clean, customisable interface; and 4) notifies instructors during potential moments of interest [?].



Figure 5: Different views of MM dashboard: SSV (left), ASV (centre), and RSV (right).

The MM dashboard has three ways of data visualisation, as we can see in 5: 1) single session view (SSV), which displays real-time learning data from a single student's session; 2) all sessions view (ASV), which shows real-time learning data from all connected students; and 3) replay session view (RSV), which allows the teacher to interact with previously recorded session data from a single student session [?]. The views are divided into panels, and we can also see that the SSV and ASV have a right-side navbar used to switch between panels [?].

However, the dashboards do come with a range of issues. For the MM dashboard to work, students must have a range of sensors, which are uncommon for them [?]. Teachers also report no experience with some of the hardware required for the MM dashboard to work. Although this is a key problem, it was not the MM dashboard’s main issue; the main issue was that teachers may lack the knowledge to understand and interpret the dashboard results, as the majority of users reported unfamiliarity with the measurements provided. The importance of the metrics was also questioned [?]. The study also noted that hands-on training may be required to help teachers use the system efficiently [?].

The limitations highlight a critical gap in the practicality and potential for utilisation of the learning dashboards. Whilst the dashboards provide high-level insights, many of these insights are difficult to understand, prompting educators to question their importance. Also, these dashboards tend to be unintuitive, which can be especially an issue for educators who are not STEM-oriented. We can leverage these insights to create clearer indicators, such as struggle, and by taking a more straightforward approach to the dashboard, we can address these gaps and create better, more useful learning dashboards.

2.5.3 Early Warning and At-Risk Student Systems

Early warning systems in education identify students at risk of poor learning outcomes by analysing routinely collected data and surface those students to instructors before failure becomes irreversible [?]. In education, an At-risk student system is an early warning system that spots struggling students before it is too late [?]. Recently, many approaches to at-risk student systems have leveraged AI to identify students at risk [?].

Student data is analysed to flag potential issues. Typically, grades, attendance, online activity, and behaviour are used to alert teachers to help students early, before problems grow [?]. The datafication of education enables the use of automated methods to detect patterns in extensive educational data. Approaches leverage predictive models to use the data, predicting the likelihood that a student will fail a course [?].

We can look at the edInsight Early Warning System as an example. This system looks at the course as a whole and uses the information gained to determine an at-risk ranking for all students [?]. The system leverages a scoring system, based on specific categories such as the Course grade category [?]:

Grade	Score
C-	2
D+	4
D	8
F	10

Table 2: Scoring system based on grades, where lower grades add to the score, showing that a student is struggling more

With the scores calculated, educators have a clear idea of which students are most at risk, making this information useful for directing attention to those who need the most help [?]. The same approach has been validated in academic environments. Arnold and Pistilli describe the Course Signals system at Purdue, which is an early intervention system that combines grade, past academic history and demographic predictors into a traffic light indicator for instructors, reporting measurable gains in student retention [?]. Jayaprakash et al. generalises this further with an open-source initiative, the Open Academic Analytics Initiative (OAAI), which has been deployed across multiple US institutions, demonstrating that the underlying predictive techniques are not specific to a single platform [?].

Although the system is beneficial for identifying students who are struggling at the course level, it does not necessarily help identify students at the lab level, as its design assumptions do not align with the

demands of lab sessions. Educators working in real-time sessions require live data. Hence, they have immediate awareness of which students are currently struggling, rather than future failure. However, we can apply the same concepts used in at-risk systems to identify students who are struggling the most and questions that have the most difficulty. As a result, as a result while traditional at risk systems are not designed for lab sessions, the underlying principles can be adapted to meet the needs of real-time lab analytics.

2.6 Research Gaps

This subsection reviewed a range of relevant literature. Learning analytics has proved incredibly useful in higher education by providing data on MOOC completion rates, click counts, and time spent, which have significantly aided pedagogical decision-making. Learning dashboards have also emerged as the primary output of learning analytics, enabling educators to gain insight into how different students are performing through an intuitive user interface. Real-time analytics has enabled processing data as it arrives, benefiting multiple sectors. Although all this work has proven highly beneficial in education and other sectors where data use is extremely valuable, there are various gaps we will address in this thesis.

In particular, computer labs face a distinct challenge that existing approaches don't address well. In these settings, teachers and assistants have no way to swiftly identify those who need help and must rely on those who put their hands up. The problem with this is that it does not account for shy students who may not want to put their hands up, causing them to spend too much time on a question. Also, current dashboards focus on data collected towards the end of the lab; this data may not be as valid afterwards. By creating a dashboard that uses real-time analytics, we can efficiently leverage this data to support students during lessons.

Furthermore, there has been no use of AI to classify data and provide a new metric, which could remove the need for educators to read every solution. This metric can be used to assess how close the student is to the correct answer and can provide a valuable indicator of how much a student is struggling. Combining this with other metrics, such as attempts and time spent, will help improve student outcomes. Research on struggle detection highlights that existing approaches operate on a course level rather than within live sessions and rely solely on threshold-based criteria that do not capture most factors that affect student difficulty. Furthermore, whilst collaborative filtering has been explored as a similarity-driven alternative to the scoring method in the educational context, its application to real-time lab struggle detection remains unaddressed due to constraints imposed by data sparseness and unreliability at the start of sessions. There is also a lack of dashboards that integrate with smart devices in computer labs. This can be used to send data from the dashboard to lab assistants' smart devices, such as phones, watches, or even glasses, to indicate which students they can help. This motivates the development of a real-time learning analytics dashboard that supports instructor awareness and prioritisation through smart device integration.

2.7 Requirements Specification

2.7.1 Functional Requirements

In this section, we define the core functionality required of the proposed system to address the problem in Chapter 1 and the gaps highlighted earlier. These requirements specify what the system must do regardless of implementation.

FR1: Live ingestion of Lab data The system must be capable of ingesting student interaction data as it is generated in the lab sessions, which includes data which is produced whilst students are answering questions, such as attempts amount, how wrong the answer is, how long spent on the question, which

will be used for measuring student struggle as well as metrics we will use in measuring question difficulty such as total attempt amount, total correct submissions and average time spent.

FR2: Computation of student struggle The system must compute a metric to represent how much a student is struggling in labs, derived from data highlighted in **FR1**. This representation should represent short-term difficulty rather than long-term ability and predicted outcomes.

FR3: Computation of question difficulty The system must compute a metric for question difficulty used to represent how hard a question or task is in a lab session, derived from the data highlighted in **FR1**. This will represent a task that is difficult for everyone in attendance at the lab.

FR4: Real-time prioritisation The system must provide an indicator of which students are struggling most in computer labs, based on student struggle data, allowing instructors to aid those who need it most.

FR5: Present Relevant Analytics The system must present relevant analytics for the labs in a suitable, intuitive form that can be interpreted quickly to enable quick decision-making.

FR6: Smart Device The system must integrate with smart devices such as phones, watches and glasses, in particular the smartphones so that information can be sent swiftly to lab assistants.

FR7: Lab Assistant Ranking System The system will record and display lab assistant contributions during the session so that assistant activity can be monitored and incentivised.

2.7.2 Non-Functional Requirements

In addition to functional requirements, the system must satisfy non-functional requirements to ensure the system operates as intended. These requirements reflect the practical constraints of the live lab sessions and ensure efficient operation.

NFR1: Real Time Performance The system must process and update analytics with minimal latency to support swift decision-making in lab sessions. Delays should not prevent an instructor from responding to students who need help.

NFR2: Interpretability Outputs should be interpretable by instructors without requiring specialist knowledge or extensive training. The system should be intuitive so that it can be used across all educational disciplines. Analytics should aid in understanding rather than providing overly complex information.

NFR3: Robustness to noisy data The presence of noisy or incomplete data should not interfere with the system's operations. Noisy or incomplete data should be accounted for and not interfere with the provision of correct analytics.

NFR4: Scalability The system should be capable of operating efficiently in various labs regardless of module, class size and question type.

NFR5: Privacy and Ethical Data must be handled responsibly and solely for the purpose of providing analytics in computer labs, and it must not be used for any purpose other than aiding students.

NFR6: Extensibility and Maintainability The system must be designed to accommodate additional data sources or metrics without requiring a fundamental redesign, enabling future extensions.

2.7.3 Requirements Prioritisation (MoSCoW)

For this section, we will use a MoSCoW table to highlight the project's requirements prioritisation, reflecting on time constraints and the project scope.

Must Have	Should Have	Could Have	Won't Have
FR1	FR5	FR7	Long term student outcomes
FR2	FR6		Fully AI decision making
FR3	NFR3		Avatar based AI assistance
FR4	NFR4		
NFR1	NFR6		
NFR2			
NFR5			

Table 3: MoSCoW table highlighting requirements for proposed software

3 Design and Modelling

This section describes the design and architecture of the lab dashboard, highlighting how various components are implemented. It covers the data endpoint, defining what data enters the system and in what form, the mathematical models used to derive struggle and difficulty and the user interface design which exposes the analytics to educators.

3.1 System Architecture

The proposed system adopts a layered architecture designed to support real-time learning analytics in computer lab environments. The architecture separates data generation, data ingestion and processing, as well as decision-making and action, into three distinct layers, ensuring each component can evolve independently.

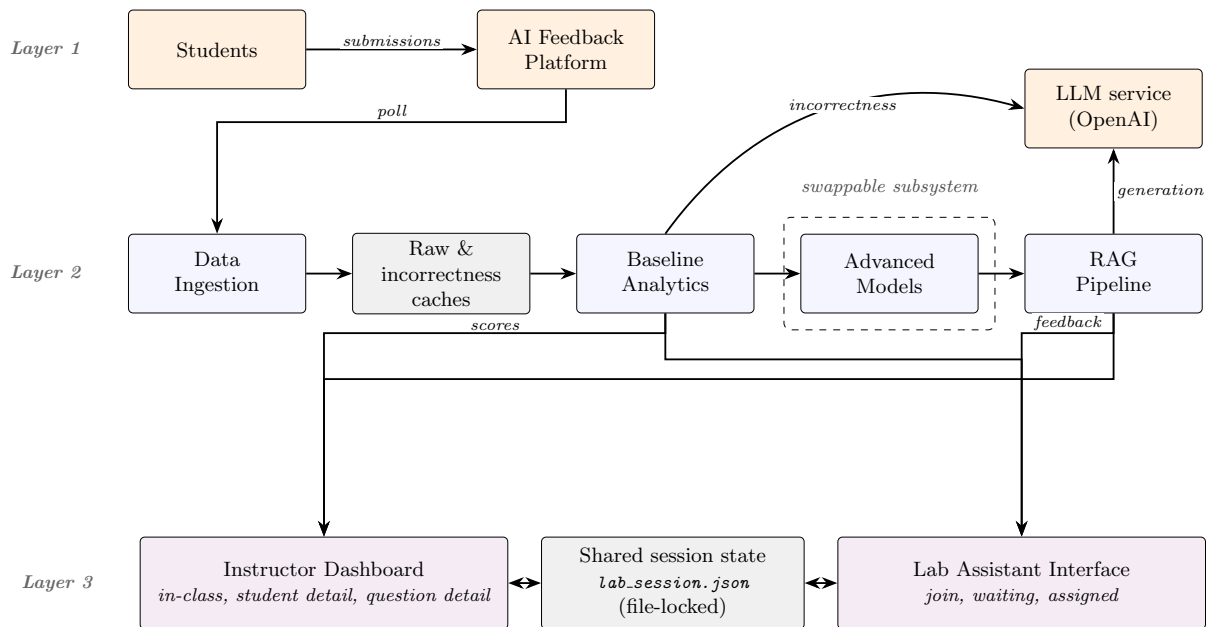


Figure 6: V2 system architecture: a three-layer pipeline from data ingestion through baseline and advanced analytics and the RAG pipeline to the instructor and lab-assistant surfaces, coordinated via the file-locked shared session state.

Data Generation Layer As seen in Figure 6, student interactions form the primary source of data. When students submit an attempt or request feedback, the interaction data is recorded at the endpoint and not modified.

Data Ingestion and Processing Layer Data ingestion is performed periodically according to the user-defined refresh rate. Retrieved data will be stored in the raw data store, then parsed and normalised before being used for calculations. The data will be used to determine student struggle, question difficulty, and other required metrics.

Decision and Action Layer Data Analytics will now be sent to the learning dashboard for presentation to the instructor, thereby aiding decision-making. The instructor will identify students who are struggling.

More complex questions will also be identified so that the instructor can go through the questions with the whole class. Instructors will use the dashboard to assign tasks to assistants to assist students who need help.

This layered design supports the project's aim of providing timely, targeted support during lab sessions while using the system as a decision-making tool for student support. Later sections build on this architecture.

3.2 Data Endpoint

A data endpoint is a specific URL or address in a web service or API that defines where and how clients can access the service. The proposed system integrates with the existing AI feedback platform through a single external data endpoint. The data endpoint is crucial in our system as it provides access to detailed records of student submissions and feedback interactions generated as students engage with lab questions. As outlined in Figure 6, the endpoint serves as the entry point to the analytics pipeline, supplying the raw data that will be processed, normalised, and aggregated for data visualisation. The integration is designed to be read-only, ensuring the data and metrics are not tampered with.

The endpoint returns an array in JSON format, where each record corresponds to a specific session based on the session's first submission time. Alongside standard fields such as module, question, timestamp and user, each record has an XML payload. The payload captures the interaction history for a given session, including submissions and AI responses. The design reflects the constraints of the existing system while providing vital information for our analytics.

Interaction data are organised around sessions, where a session represents a continuous period of student engagement from the first submission attempt. Within a session, students may have multiple submissions, getting multiple AI responses. This is beneficial in reflecting how their answers may improve question by question. The structure allows the capture of temporal patterns, repeated attempts, and help-seeking behaviour, which will be vital for identifying short-term struggle and difficult questions in lab sessions.

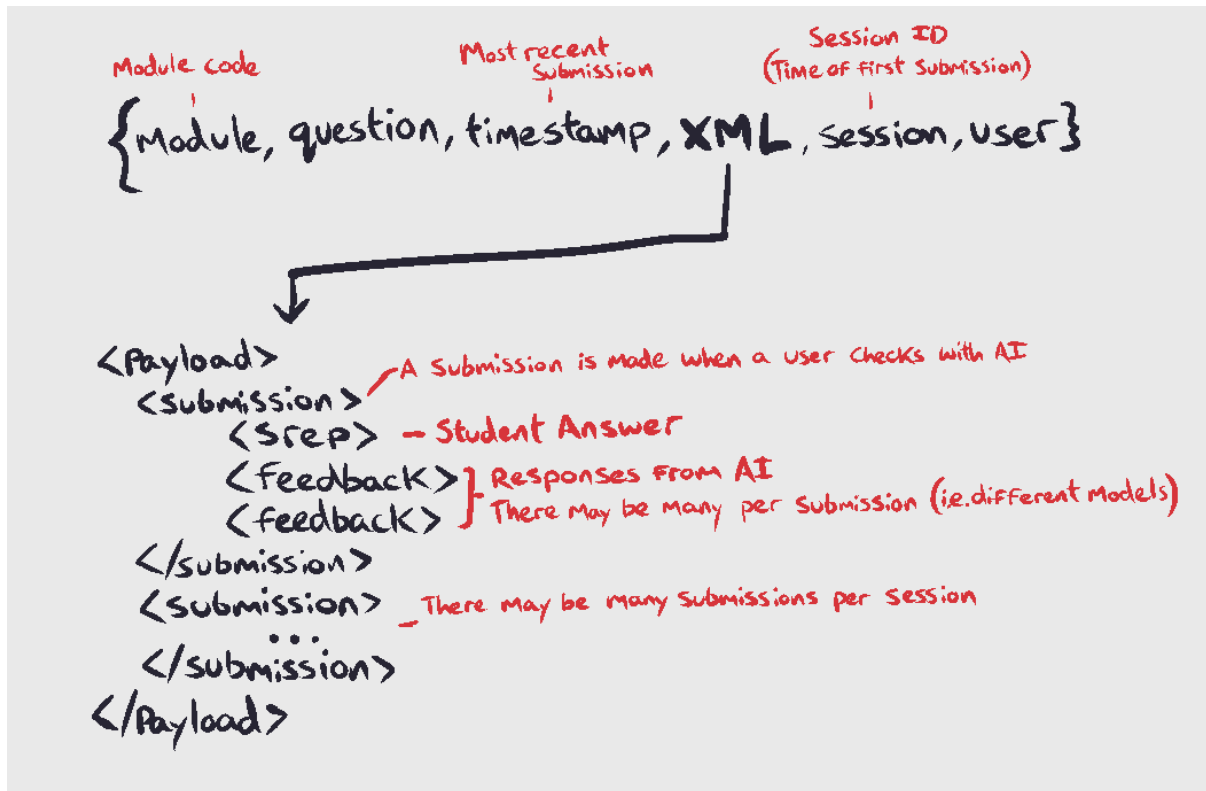


Figure 7: Structure of session-based interaction data returned by feedback system

From the session-based data, the system derives a set of core metrics for real-time monitoring. These include attempts per question, time between attempts, AI-based indicators of correctness, and the frequency with which students request feedback. By using these, we will enable the system to distinguish among students and identify those who are struggling the most. The derived metrics will form the basis of our two key metrics: student struggle and question difficulty.

It is essential to consider that we may receive incomplete or noisy data during sessions, which could stem from students repeatedly returning to the AI feedback system or from procrastination on their work. Not all indicators of struggle and question difficulty are valuable on their own; hence, they are combined to produce indicators of student struggle and question difficulty.

3.3 Baseline Analytics Design

This section describes the baseline metrics, student struggle and question difficulty and the data from the endpoint that is used to derive them.

3.3.1 Student Struggle Model

Modelling Objective To model student struggle, let s denote a student and σ denote the session. We will denote students who struggle with a scalar function:

$$S(s, \sigma) \in [0, 1] \quad (5)$$

The student struggle function will allow us to quantify the degree of difficulty that students s are currently experiencing during session σ , based on the observable data from the endpoint in Section 3.2.

The model will support real-time prioritisation in computer labs.

Identified Variables From the session-based interaction data described in Section 3.2. We can derive the following quantities:

- $n_{s,\sigma}$ - The total number of submissions made by a student s during session σ
- $t_{s,\sigma}$ - The total active time elapsed during session σ
- $i_{s,\sigma}$ - The average AI incorrectness measure for student s during session σ , derived from AI-Based answer ratings
- $r_{s,\sigma}$ - The number of retry submissions within session σ
- $d_{s,\sigma}$ - The regression slope of incorrectness scores against the submission index, which captures whether a student is improving or deteriorating across the session
- $\text{rep}_{s,\sigma}$ - The number of exact-match repeated submissions within session σ

By leveraging metrics linked to student struggle, we can produce a student struggle score

AI-Based Answer Rating Let $A_{s,\sigma,k}$ be the k -th submitted answer by student s in session σ . We will derive an LLM-based scoring function. Where Q_σ is the current question and A_σ is the answer, we will define our LLM-based scoring function to be:

$$c_{s,\sigma,k} = C(A_{s,\sigma,k}, A_\sigma, Q_\sigma) \in [0, 1] \quad (6)$$

where $c_{s,\sigma,k}$ is the correctness measure that measures the quality of a students answer, where 1 is the correct answer and 0 is completely wrong. We can then define our incorrectness measure:

$$i_{s,\sigma,k} = 1 - c_{s,\sigma,k} \in [0, 1] \quad (7)$$

Normalisation The retry rate, repetition rate and trajectory slope are derived from the counts identified earlier:

$$\tilde{r}_{s,\sigma} = \frac{r_{s,\sigma}}{n_{s,\sigma}} \in [0, 1], \quad \text{rep}_{s,\sigma} = \frac{\text{rep}_{s,\sigma}}{n_{s,\sigma}} \in [0, 1].$$

The trajectory slope $d_{s,\sigma}$ is signed (negative values indicate improvement, positive values indicate deterioration) and therefore unbounded by definition. The mean incorrectness $i_{s,\sigma}$ and the time decayed recent-incorrectness $A^{\text{raw}}(s, \sigma)$ are already bounded to $[0, 1]$ by construction as each is an average of values already in $[0, 1]$.

To ensure each component contributes the weighting intended, every signal is min-max normalised across the cohort for the session with:

$$\hat{x}_{s,\sigma} = \frac{x_{s,\sigma} - x_{\min}}{x_{\max} - x_{\min}}, \quad x_{\min} = \min_{s' \in S} x_{s',\sigma}, \quad x_{\max} = \max_{s' \in S} x_{s',\sigma}.$$

Applied to the seven signals introduced earlier, we have:

$$\hat{n}_{s,\sigma}, \hat{t}_{s,\sigma}, \hat{i}_{s,\sigma}, \hat{r}_{s,\sigma}, \widehat{A^{\text{raw}}}(s, \sigma), \hat{d}_{s,\sigma}, \widehat{\text{rep}}_{s,\sigma} \in [0, 1].$$

Min-max normalisation is applied even to signals already bounded to $[0, 1]$, because the configured weights only match the effective contribution of each signal when every signal occupies the full $[0, 1]$ range, otherwise a signal that clusters in a sub-interval would contribute less than the weight implied. When $x_{\min} = x_{\max}$, the normalised value is conventionally set to 0.

Recent submissions weighted by time decay To reflect that recent submissions are more indicative of current struggle than older submissions in a live setting, each submission is weighted by an exponential decay in the time elapsed since it was made, rather than its position in its submission sequence. Let $\Delta_{s,\sigma,k}$ denote the elapsed time in seconds between submission k and the current refresh. The time-decay weight is defined as:

$$w_{s,\sigma,k} = \exp\left(-\frac{\ln 2}{H} \Delta_{s,\sigma,k}\right), \quad (8)$$

where $H = 1800$ s is the half-life; a submission made 30 minutes ago contributes half the weight of a submission made just now. The recent-incorrectness signal is then the time-decay weighted mean of the per-submission incorrectness:

$$A^{\text{raw}}(s, \sigma) = \frac{\sum_{k=1}^{n_{s,\sigma}} w_{s,\sigma,k} i_{s,\sigma,k}}{\sum_{k=1}^{n_{s,\sigma}} w_{s,\sigma,k}} \in [0, 1]. \quad (9)$$

Time-based decay is preferred over position-based weighting because students submit at irregular intervals. Weighting by elapsed time gives a more accurate reflection of current struggle rather than submission position. Since each $i_{s,\sigma,k} \in [0, 1]$, and the weights are positive, the convex combination $A^{\text{raw}}(s, \sigma)$ is bounded in $[0, 1]$ by construction, and does not require normalisation.

Struggle Score Definition The raw student struggle is a convex combination of the seven bounded components:

$$S^{\text{raw}}(s, \sigma) = \alpha \hat{n}_{s,\sigma} + \beta \hat{t}_{s,\sigma} + \gamma \hat{i}_{s,\sigma} + \delta \hat{r}_{s,\sigma} + \eta \widehat{A^{\text{raw}}}(s, \sigma) + \zeta \hat{d}_{s,\sigma} + \theta \widehat{\text{rep}}_{s,\sigma}, \quad (10)$$

where the coefficients are non-negative and sum to one:

$$\alpha, \beta, \gamma, \delta, \eta, \zeta, \theta \geq 0, \quad \alpha + \beta + \gamma + \delta + \eta + \zeta + \theta = 1.$$

The default values used in the deployed system are:

$$\alpha = 0.10, \quad \beta = 0.10, \quad \gamma = 0.20, \quad \delta = 0.10, \quad \eta = 0.38, \quad \zeta = 0.05, \quad \theta = 0.07,$$

summarised by signal in Table 4. The recent-incorrectness signal carries the largest weight ($\eta = 0.38$) because the model prioritises within-session struggle over cumulative behaviour. The longer tail signals (trajectory and repetition) carry small but non-zero weights so they can break ties between students who look similar on the headline metrics. The weights were tuned by trial and error in the absence of labelled ground-truth data. Since every component is bounded in $[0, 1]$ and all the weights are convex,

$$S^{\text{raw}}(s, \sigma) \in [0, 1].$$

Symbol	Meaning	Weight
$\hat{n}_{s,\sigma}$	Submission count (min-max normalised)	$\alpha = 0.10$
$\hat{t}_{s,\sigma}$	Active time (min-max normalised)	$\beta = 0.10$
$\hat{i}_{s,\sigma}$	Mean LLM incorrectness (min-max normalised)	$\gamma = 0.20$
$\hat{r}_{s,\sigma}$	Retry rate (min-max normalised)	$\delta = 0.10$
$\widehat{A^{\text{raw}}}(s, \sigma)$	Time-decayed recent incorrectness (min-max normalised)	$\eta = 0.38$
$\hat{d}_{s,\sigma}$	Improvement-trajectory slope (min-max normalised)	$\zeta = 0.05$
$\widehat{\text{rep}}_{s,\sigma}$	Answer-repetition rate (min-max normalised)	$\theta = 0.07$

Table 4: Seven signals contributing to the baseline student struggle score $S^{\text{raw}}(s, \sigma)$, with default weights; weights sum to 1.00

Bayesian shrinkage towards the class mean A student with very few submissions can produce a S^{raw} that is dominated by noise; a student with two submissions may register a 100% retry rate purely because the second submission was a repeat of a question, without that being indicative of struggle. To stabilise scores when the per-student observation count is small, S^{raw} is shrunk toward the class mean with a weight that depends on the student's submission count $n_{s,\sigma}$:

$$S^{\text{shrunk}}(s, \sigma) = \frac{n_{s,\sigma}}{n_{s,\sigma} + K} S^{\text{raw}}(s, \sigma) + \frac{K}{n_{s,\sigma} + K} \bar{S}_\sigma^{\text{raw}}, \quad (11)$$

where $\bar{S}_\sigma^{\text{raw}}$ is the cohort mean of the raw struggle in session σ and K is a hyper-parameter controlling the strength of the shrinkage. The default value is $K = 5$, so a student with $n_{s,\sigma} \gg 5$ submissions is essentially unaffected while a student with $n_{s,\sigma} \ll 5$ submissions is shrunk close to the class mean. The technique is the classical empirical Bayes shrinkage estimator in the tradition of James-Stein and Efron & Morris [?], here in the conjugate-prior form in which K acts as a prior count [?]. It is applied here at the level of the aggregate struggle score rather than at the level of the individual signals. Shrinkage and temporal smoothing serve complementary roles, where shrinkage handles cold-start uncertainty and temporal smoothing handles noise when refreshing scores within a session.

Temporal updating for real-time operation To provide a responsive, stable indicator, we will update struggle with exponential smoothing:

$$S_t(s, \sigma) = (1 - \lambda)S_{t-1}(s, \sigma) + \lambda S^{\text{shrunk}}(s, \sigma), \quad \lambda \in (0, 1] \quad (12)$$

It follows that the update preserves boundaries, giving:

$$S_t(s, \sigma) \in [0, 1], \quad \forall t$$

Interpretation of model Higher values of $S_t(s, \sigma)$ indicate that student s is experiencing sustained difficulty in the current session. The metric in Equation 12 will prove to be valuable in real-time prioritisation of help in lab sessions. This measure does not indicate student ability in the cohort but rather in lab struggle.

3.3.2 Temporal Smoothing

Live struggle scores carry noise from two sources. Within a single score computation, submissions arrive at irregular intervals, so recent submissions should count more than older ones. Across refresh cycles, a student's score can change even when nothing meaningful has happened. Both techniques were introduced in the previous section; this subsection names them explicitly and contrasts their roles.

Per-submission time decay inside A^{raw} The first mechanism reweights submissions within a single computation of S^{raw} . Each submission contributes to the recent-incorrectness signal $A^{\text{raw}}(s, \sigma)$ with a weight that decays exponentially with the time elapsed since the submission was made, as defined in Equations 8 and 9. The half-life is fixed at $H = 1800$ s, so a submission made 30 minutes ago contributes half the weight of a submission made just now. Because students submit at irregular intervals, time-based decay gives a more accurate reflection of current struggle than index-based weighting.

Exponentially Weighted Moving Average (EWMA) across refresh cycles applied to S^{shrunk} The second mechanism stabilises the score in between refresh cycles. After the raw struggle score is computed and shrunk toward the cohort mean (Equation 11), the resulting S^{shrunk} is smoothed across refresh cycles using a standard exponentially weighted moving average (EWMA), as given by Equation 12.

The smoothing rate $\lambda = 0.3$ is fixed by the configuration constant `SMOOTHING_ALPHA`, gated on by `SMOOTHING_ENABLED` (which will be left active by default but can be toggled off). A small λ trusts past observations more, giving a smoother trajectory, while a larger λ trusts the current shrunk score more, giving a more responsive trajectory. The chosen value of $\lambda = 0.3$ was tuned by trial and error to give a good balance between responsiveness and stability, in the absence of labelled ground-truth data.

Property	Per-submission decay	EWMA across refresh
What it smooths	Submissions inside one A^{raw} computation	The final score S across refresh cycles
Granularity	Per submission (seconds)	Per refresh tick (typically every few seconds)
Decay parameter	Half-life $H = 1800$ s	Smoothing rate $\lambda = 0.3$
Recurrence form	Weight $\exp(-\ln 2/H \cdot \Delta)$ on each submission	$S_t = (1 - \lambda)S_{t-1} + \lambda S^{\text{shrunk}}$
Noise it suppresses	Old submissions dominating a still-active student	Score fluctuations across refresh ticks
Source of the problem	Irregular submission intervals within one session	Single-submission fluctuations between refreshes

Table 5: The two temporal-smoothing mechanisms in the baseline struggle model and the role each plays.

3.3.3 Question Difficulty Model

Modelling Objective To model question difficulty, let q denote a question during the lab session, and let $\mathcal{S}_q$ denote the set of student-session pairs associates with task q . We will denote questions that are difficult with a scalar function:

$$D(q) \in [0, 1] \quad (13)$$

The question difficulty function will allow us to quantify the degree of difficulty of a question q , that the students in the $\mathcal{S}_q$ set are finding the hardest, based on the observable data from the endpoint in Section 3.2. The model will support real-time prioritisation in computer labs

Identified Variables From the session based interaction data described in Section 3.2, the following quantities are derived. A submission is treated as correct if its LLM-derived incorrectness score $i_{s,\sigma,k}$ is below a fixed threshold of τ . The system uses $\tau = 0.5$ by default.

- n_q - The total number of attempts on question q
- c_q - The total number of correct attempts on question q
- t_q - The average time spent on question q
- a_q - The average number of attempts spent on question q
- f_q - Average AI incorrectness measure for q , derived from AI-Based answer ratings
- p_q - The number of students whose first attempt at q was incorrect

By leveraging metrics linked to question difficulty, we can produce a question difficulty score.

Normalisation The incorrect rate and first-attempt failure rate are derived from the counts identified above:

$$\tilde{c}_q = 1 - \frac{c_q}{n_q} \in [0, 1], \quad \tilde{p}_q = \frac{p_q}{|\mathcal{S}_q|} \in [0, 1],$$

The average time and average attempts are min-max normalised across the cohort of questions in the current session:

$$\tilde{t}_q = \frac{t_q - t_{\min}}{t_{\max} - t_{\min}} \in [0, 1], \quad \tilde{a}_q = \frac{a_q - a_{\min}}{a_{\max} - a_{\min}} \in [0, 1].$$

The average LLM incorrectness f_q is already bounded by construction, so:

$$\tilde{f}_q = f_q \in [0, 1].$$

Difficult Task Definition Question difficulty is defined as a convex combination of the five bounded components:

$$D^{\text{raw}}(q) = \alpha \tilde{c}_q + \beta \tilde{t}_q + \gamma \tilde{a}_q + \delta \tilde{f}_q + \epsilon \tilde{p}_q, \quad (14)$$

where the coefficients are non-negative and sum to one:

$$\alpha, \beta, \gamma, \delta, \epsilon \geq 0, \quad \alpha + \beta + \gamma + \delta + \epsilon = 1.$$

The default values used in the deployed system are:

$$\alpha = 0.28, \quad \beta = 0.12, \quad \gamma = 0.20, \quad \delta = 0.20, \quad \epsilon = 0.20,$$

summarised by signal in Table 6. Since every component is bounded in $[0, 1]$ and the combination is convex:

$$D^{\text{raw}}(q) \in [0, 1].$$

Symbol	Meaning	Weight
$\tilde{c}_q$	Incorrect rate, $1 - c_q/n_q$	$\alpha = 0.28$
$\tilde{t}_q$	Average time per student (min-max normalised)	$\beta = 0.12$
$\tilde{a}_q$	Average attempts per student (min-max normalised)	$\gamma = 0.20$
$\tilde{f}_q$	Average LLM incorrectness score	$\delta = 0.20$
$\tilde{p}_q$	First-attempt failure rate, $p_q/ \mathcal{S}_q $	$\epsilon = 0.20$

Table 6: Five signals contributing to the baseline question difficulty score $D^{\text{raw}}(q)$, with default weights; weights sum to 1.00.

Temporal updating for real-time operation To provide a responsive stable indicator, we will update question difficulty with exponential smoothing across refresh cycles:

$$D_t(q) = (1 - \lambda) D_{t-1}(q) + \lambda D^{\text{raw}}(q), \quad \lambda \in (0, 1]. \quad (15)$$

Interpretation of Model Higher values of $D_t(q)$ indicate that a question is difficult for the class as a whole during the current session. The metric will prove to be valuable in real-time prioritisation of help in lab sessions.

3.3.4 Alternative Modelling Approach: Collaborative Filtering

Motivation The parameter-based struggle model defined in Section 3.3.1 assigns a score to each student independently base on a weighted combination of normalised interaction features. Whilst this approach is computationally lightweight and interpretable, it depends on the weightings $\alpha, \beta, \gamma, \delta, \eta$. We can use an alternative modelling approach based on Collaborative Filtering, where rather than scoring each student in isolation, we explore similarity between students.

Student Interaction Matrix Let $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$ denotes the set of students present in sessions σ . For each student s_i , we define a five-dimensional feature vector derived from the normalised interaction quantities introduced in Section 3.3.1:

$$\mathbf{x}_{s_i} = (\hat{n}_{s_i, \sigma}, \hat{t}_{s_i, \sigma}, \hat{e}_{s_i, \sigma}, \hat{f}_{s_i, \sigma}, A^{\text{raw}}(s_i, \sigma))^{\top} \in [0, 1]^5 \quad (16)$$

Collecting these vectors row-wise, we define the student interaction matrix:

$$\mathbf{X} \in [0, 1]^{n \times 5}, \quad \mathbf{X}_{ij} = [\mathbf{x}_{s_i}]_j \quad (17)$$

Each row of $\mathbf{X}$ encodes one student's current behavioural profile during the session. The matrix is updated at each refresh interval as new data arrives from the endpoint.

Student Similarity Measure To quantify how similar two students are, we apply cosine similarity over their feature vectors. Cosine similarity is preferred over Euclidean distance because it measures the angle between two vectors rather than their absolute magnitudes, making it robust to differences in overall activity levels between students.

$$\text{sim}(s_i, s_j) = \frac{\mathbf{x}_{s_i}^{\top} \mathbf{x}_{s_j}}{\|\mathbf{x}_{s_i}\|_2 \cdot \|\mathbf{x}_{s_j}\|_2} \in [0, 1] \quad (18)$$

Since all components of $\mathbf{x}_{s_i}$ are normalised to $[0, 1]$, the dot product is non-negative, and cosine similarity is bounded to $[0, 1]$. This yields a symmetric similarity matrix $\mathbf{W} \in [0, 1]^{n \times n}$ where $W_{ij} = \text{sim}(s_i, s_j)$ and $W_{ii} = 1$ by definition. A value close to 1 indicates near identical behavioural profiles, whilst a value close to 0 indicates students with very different profiles.

Help-Seeking as a Proxy for Struggle In traditional systems, user ratings serve as the signal across similar users. In the lab setting, no explicit ratings are available, so we define a binary help seeking indicator derived from the parametrix model as a proxy:

$$h_{s, \sigma} = \begin{cases} 1 & \text{if } S_t(s, \sigma) \geq \tau \\ 0 & \text{otherwise} \end{cases} \quad (19)$$

where $\tau \in [0, 1]$ is a threshold above which a student is considered to be observably struggling based on their score. This allows collaborative filtering to leverage signals from students who are struggling with similar peers who may not have crossed the threshold, enabling earlier intervention.

Predicted Help Need with k -Nearest Neighbours For a student s for whom $h_{s, \sigma}$, we estimate their latent help need by aggregating the help-seeking behaviour of their k most similar peers. Let $\mathcal{N}_k(s)$ denotes the set of k students with the highest similarity to s , excluding s :

$$\mathcal{N}_k(s) = \underset{s' \in \mathcal{S} \setminus \{s\}, |\mathcal{N}|=k}{\text{arg top-}k} \text{sim}(s, s') \quad (20)$$

The predicted help-need score is then defined as a similarity-weighted average of the help indicators of the k nearest neighbours.

$$\hat{h}_{s,\sigma} = \frac{\sum_{s' \in \mathcal{N}_k(s)} \text{sim}(s, s') \cdot h_{s',\sigma}}{\sum_{s' \in \mathcal{N}_k(s)} \text{sim}(s, s')} \in [0, 1] \quad (21)$$

A high value of $\hat{h}_{s,\sigma}$ indicates that student s is similar to peers who required help, suggesting they may benefit from intervention.

Comparison with Parameter-Based Model Table 7 summarises the key structural differences between the adopted parametric model and the collaborative filtering alternatively, highlighting the properties that informed the choice of primary approach.

Property	Parameter-Based Model	Collaborative Filtering
Scoring basis	Weighted formula	Peer similarity
Interpretability	High	Moderate
Cold start	Immediate	Requires $\geq k + 1$ active students
Computational complexity	$O(n)$ per update	$O(n^2)$ per update

Table 7: Structural comparison of the parametric struggle model and the collaborative filtering alternative

Justification for Current Approach The parameter-based model is adopted as the primary approach for several reasons. This approach does not require a minimum class size to be considered viable, making it robust to sessions with few students where the similarity matrix $\mathbf{W}$ would be unreliable. Additionally, it produces more reliable results immediately, whereas a collaborative filtering approach produces results that are not reliable at the start of a session.

The collaborative filtering approach is implemented alongside the parameter-based model as a secondary indicator, exposed through the Settings panel as a user-toggle option like Bloomberg’s terminal; it is enabled by default with $k = 3$ nearest neighbours and a configurable elevation threshold, and can be turned off if the instructor prefers the parametric model.

3.3.5 Mistake Clustering

Beyond aggregate difficulty, instructors benefit from seeing what kind of mistakes students are making on a given question. The system clusters incorrect submissions per question so that dominant error patterns are surfaced rather than left hidden behind a single “incorrect” count. The pipeline combines three standard text-mining techniques - TF-IDF vectorisation, k -means clustering, and silhouette-based model selection - followed by an LLM labelling step that attaches human-readable descriptions to the clusters. The formal definitions of TF-IDF [?, ?], k -means [?, ?], and the silhouette coefficient [?] are described in Section 2.3.2; this section describes the design choices specific to the lab setting.

Pipeline For each question q , the incorrect submissions (those whose LLM-derived incorrectness score $i_{s,\sigma,k}$ is above the threshold τ) are vectorised using TF-IDF over their answer text. k -means clustering is then applied for each k in a small range, and the value that maximises the mean silhouette coefficient is selected automatically, removing the need for the instructor to specify a target cluster count.

$$k^* = \arg \max_{k \in \{2, \dots, K_{\max}\}} \frac{1}{|X_q|} \sum_{x \in X_q} s(x; k), \quad (22)$$

Once clusters are formed, a generative model is prompted with a sample of representative answers from each cluster to produce a short natural language label [?], which is displayed alongside the cluster in the Question Detail View.

Parameters and Gating Clustering is only attempted when at least $N_{\min} = 3$ incorrect submissions are available for question q ; below this threshold the noise in cluster assignments outweighs any pattern they might reveal, and the question is shown without clusters. The silhouette sweep evaluates $k \in \{2, \dots, K_{\max}\}$ with $K_{\max} = 5$; mistake patterns within a single lab question rarely partition into more than five qualitatively distinct groups. For each surfaced cluster, up to three representative answers are shown in the UI so the instructor can verify the cluster’s coherence at a glance.

Interpretation Where the question difficulty score $D_t(q)$ indicates whether a question is causing trouble, the mistake clustering provides insight into what kind of trouble it is causing. This complements the parametric difficulty signal: a question may register as “Medium” overall but reveal a single dominant error pattern affecting most struggling students, in which case the appropriate intervention is to address that misconception with a class-wide explanation rather than helping students one-on-one.

3.4 Advanced Model Design

3.4.1 Measurement Confidence

Every incorrectness score $i_{s,\sigma,k}$ in this design is derived from an LLM judgement that is uncertain. To prevent the instructor from over-interpreting unreliable scores, each submission also carries a confidence measurement value $\kappa_{s,\sigma,k} \in [0, 1]$ that quantifies how trustworthy the corresponding incorrectness estimate is, in the spirit of classical test theory [?].

The confidence is composed of two factors and a base multiplier:

$$\kappa_{s,\sigma,k} = \kappa_0 \cdot \text{length}(A_{s,\sigma,k}) \cdot \left(\frac{1}{2} + \frac{1}{2} \text{extremity}(i_{s,\sigma,k}) \right), \quad (23)$$

where $\kappa_0 \in [0, 1]$ is a base confidence value and each of the two factors is bounded in $[0, 1]$. The length factor penalises very short submissions for which the model has little signal to work with; the extremity factor rewards scores near the endpoints 0 and 1 over middling values, reflecting that the model is most certain when an answer is clearly correct or wrong and least certain when an answer is borderline. Submissions with empty or missing feedback are assigned $\kappa = 0$. The half-and-half scaling on the extremity term ensures that valid submissions with mid-range scores still receive at least half of the base confidence, prioritising smooth degradation over hard cut-offs.

In the dashboard, κ is surfaced as a coloured indicator next to the incorrectness value in the Question Detail view: green when κ exceeds a high threshold (trust the score), amber for intermediate values (verify before acting), and grey when κ falls below a low threshold (the score should not drive intervention decisions on its own). This makes the uncertainty visible to the instructor without requiring them to read the underlying score.

3.4.2 Item Response Theory

The baseline difficulty score $D_t(q)$ defined in Section 3.3.3 is a heuristic combination of observable metrics; however, it does not separate the difficulty of the question from the ability of the students who attempted it. A question registering as hard may have attracted a weaker subset of the cohort. To make this distinction, the dashboard offers an alternative difficulty score based on Item Response Theory (IRT) [?], where question difficulty and student ability are estimated jointly as latent variables.

Rasch 1-parameter logistic model The simplest IRT model is the Rasch model [?], which assigns each student s a latent ability $\theta_s \in \mathbb{R}$ and each question q a latent difficulty $\beta_q \in \mathbb{R}$. The probability of student s answering question q correctly is the sigmoid of the ability-difficulty gap:

$$P(X_{s,q} = 1 \mid \theta_s, \beta_q) = \sigma(\theta_s - \beta_q) = \frac{1}{1 + e^{-(\theta_s - \beta_q)}}, \quad (24)$$

where $X_{s,q} \in \{0, 1\}$ is the binary response (correct or incorrect). A larger β_q pushes the probability of correctness down; a larger θ_s pushes it up.

Fitting via joint maximum likelihood To estimate (θ_s, β_q) from observed responses, the binary response matrix X is first built from the session data: for each student-question pair, the best (lowest-incorrectness) attempt is selected and coded $X_{s,q} = 1$ iff the incorrectness score falls below the threshold τ . Rows and columns with too few observations are dropped to ensure stable estimation. The Bernoulli log-likelihood across the observed entries,

$$\ell(\theta, \beta) = \sum_{(s,q) \in \mathcal{O}} \left[X_{s,q} \log \sigma(\theta_s - \beta_q) + (1 - X_{s,q}) \log (1 - \sigma(\theta_s - \beta_q)) \right], \quad (25)$$

is maximised by a bounded quasi-Newton method, specifically L-BFGS-B [?], with an explicit gradient supplied for efficiency. The abilities θ are centred to zero mean at every iteration to resolve the additive identifiability degeneracy: only the difference $\theta_s - \beta_q$ enters the likelihood, so the absolute level of either parameter is unidentified without an anchor.

Mapping back to the dashboard scale The fitted difficulties $\hat{\beta}_q$ are on the logit scale and can take any real value. To make them comparable to the baseline difficulty score $D_t(q) \in [0, 1]$ and re-use the same threshold classifier, each estimate is mapped through the sigmoid:

$$D^{\text{IRT}}(q) = \sigma(\hat{\beta}_q) \in [0, 1]. \quad (26)$$

This is the value displayed on the IRT-based difficulty leaderboard and used as input to the improved struggle model in Section 3.4.4.

Two parameter extension in V2 The second iteration of the dashboard upgrades the Rasch model to the two-parameter logistic (2PL) variant of the model. Each question q now carries both a difficulty β_q and a positive discrimination parameter α_q and the response probability becomes

$$P(X_{s,q} = 1 \mid \theta_s, \alpha_q, \beta_q) = \sigma(\alpha_q(\theta_s - \beta_q)), \quad (27)$$

which recovers equation 24 when $\alpha_q = 1$. The discrimination α_q measures how sharply the question separates the ability of students, values near zero flag non-discriminating items. Identifiability now requires anchoring both location and scale: V2 enforces $\sum_s \theta_s = 0$ and $\sum_q \log \alpha_q = 0$ (equivalently that the geometric mean of α is 1). The optimiser is unchanged but α_q is reparameterised through $\log \alpha_q$ so that the positivity constraint is automatically satisfied.

Graceful degradation The joint MLE requires sufficient overlap between students and questions to converge. When the response matrix is too sparse - few questions with enough responses, or few students with enough attempts - the fit produces an empty result and the system falls back to the baseline difficulty signal. The improved struggle model builds on top of the IRT difficulty and incorporates its own weight-redistribution logic when the IRT layer is unavailable.

3.4.3 Bayesian Knowledge Tracing

The struggle and difficulty models defined so far summarise behaviour within a single snapshot of a session. They do not, however, model how a student’s understanding evolves across successive attempts. Bayesian Knowledge Tracing (BKT) [?] fills this gap by treating each (student, question) sequence as a learning trajectory under a two-state hidden Markov model: at any point the student either has or has not mastered the skill, and each attempt is an observation that updates the posterior belief over those two states.

Model Parameters Each skill (here taken as a single question) carries four parameters:

- $P(L_0)$: the prior probability that a student has mastered the skill before any attempts
- $P(T)$: the probability of transitioning from not-mastered to mastered between attempts
- $P(G)$: the probability of guessing correctly when not mastered
- $P(S)$: the probability of slipping (answering incorrectly) when mastered

All four are bounded in $[0, 1]$, with the additional identifiability constraint that $P(G) + P(S) < 1$ enforced in practice by bounding both above by 0.5.

Update Step On each attempt the system observes $X_{s,q,k} \in \{0, 1\}$, the correctness indicator derived from $i_{s,\sigma,k} < \tau$ as in Section 3.3.3. The posterior probability of mastery is updated by Bayes’ rule conditioned on the observation, and then advanced by the learning transition:

$$P(L_{k+1}) = P(L_k | X_{s,q,k}) + (1 - P(L_k | X_{s,q,k})) \cdot P(T), \quad (28)$$

where the Bayesian posterior depends on whether the attempt was correct: with probability $P(L_k)(1 - P(S))$ the student knew and did not slip, and with probability $(1 - P(L_k))P(G)$ the student did not know and guessed.

Predictive Probability Before observing the next attempt, the model’s predicted probability of correctness is

$$P(\text{correct}_{k+1}) = P(L_k)(1 - P(S)) + (1 - P(L_k))P(G), \quad (29)$$

which combines the chance of demonstrating mastery (right because they know) with the chance of guessing (right despite not knowing). This quantity is used for diagnostics (ROC-AUC against the actual outcomes) and as a per-attempt feature for downstream models.

Parameter fitting and graceful degradation Rather than fit per-skill parameters, the system estimates a single global $P(L_0), P(T), P(G), P(S)$ across all questions. The forward-algorithm log-likelihood is maximised by L-BFGS-B with the identifiability bounds described above. Fitting is skipped when fewer than a minimum number of observations are available, or when every attempt in the current view is graded the same way: with a single outcome class the learning dynamics become unidentifiable, and the system retains the previously fitted parameters rather than producing spurious estimates. Per-question and per-student mastery values feed the improved struggle model in Section 3.4.4, and an aggregate (mean mastery, minimum mastery, number of questions above a mastery threshold) is shown alongside the student’s struggle score in the Student Detail view.

Yudelson et al. [?] extend BKT with per-student parameters; the system implements only the global-parameter version for the reasons of identifiability and data sufficiency just described.

3.4.4 Improved Struggle Model

The baseline struggle score $S_t(s, \sigma)$ in Section 3.3.1 draws only on observable behaviour such as how often a student submits, how recently they have been wrong, how often they retry the same question. It does not consult the latent quantities developed in this chapter such as the student’s BKT-estimated mastery (Section 3.4.3) or the IRT-estimated question difficulty of the question they are attempting (Section 3.4.2). The improved struggle score combines all three signal groups into a single mastery-aware, difficulty-adjusted score.

Improved Struggle Score The improved struggle score combines the three components as a convex combination:

$$S^{\text{imp}}(s, \sigma) = w_{\text{beh}} \cdot B_s + w_{\text{mg}} \cdot M_s + w_{\text{da}} \cdot D_s, \quad w_{\text{beh}} + w_{\text{mg}} + w_{\text{da}} = 1, \quad (30)$$

with default weights $w_{\text{beh}} = 0.45$, $w_{\text{mg}} = 0.30$, $w_{\text{da}} = 0.25$. The behavioural component carries the largest weight because it is the most data-efficient and reliable signal, while the mastery and difficulty components provide complementary information that enhances the score’s sensitivity to underlying learning dynamics.

Behavioural composite The behavioural component B_s is the equal-weighted mean of four cohort-normalised sub-signals from the baseline model: the time-decayed incorrectness $\widehat{A}_{s, \text{raw}}$, the retry rate $\hat{r}_s$, the trajectory slope $\hat{d}_s$, and the repetition rate $\widehat{\text{rep}}_s$. Each is defined in Section 3.3.1. B_s captures the behavioural surface of struggle without consulting any latent-trait estimate.

Mastery gap The mastery gap M_s quantifies how much a student is underperforming relative to the mastery BKT attributes to them. Let $\bar{m}_s$ denote the student’s mean mastery across attempted questions, and let $1 - A_s^{\text{raw}}$ be their recent correctness rate. The gap takes the positive part of the difference between the two:

$$M_s = \max\left(\bar{m}_s - (1 - A_s^{\text{raw}}), 0\right) \in [0, 1].$$

A non-zero value indicates that the student’s recent demonstrated correctness falls below what their estimated mastery would predict; this asymmetry is often the earliest sign of transient disengagement or a momentary block. The signal is one-sided by construction: students who outperform their mastery estimate are not penalised, since this scenario reflects faster-than-expected learning rather than struggle.

Difficulty-adjusted score The difficulty-adjusted component D_s weights each inversely by the difficulty of the question that produced it. Let $\tilde{\beta}_q$ denote the normalised IRT difficulty from Section 3.4.2, and let $\mathcal{R}_s$ denote the student’s recent submission set. The score is the per-user weighted mean of the difficulty-adjusted incorrectness:

$$D_s = \frac{1}{|\mathcal{R}_s|} \sum_{k \in \mathcal{R}_s} i_{s, \sigma, k} \cdot (1 - \tilde{\beta}_q) \in [0, 1].$$

Failing an easy question contributes more to D_s than failing a hard one; failing only the hardest questions produces a near-zero score, since the cohort has already validated their difficulty. To prevent single-submission variance from dominating a sparse student’s score, the per-user mean is shrunk toward the cohort mean by the coverage ratio $|\mathcal{R}_s^{\text{covered}}|/|\mathcal{R}_s|$: a student whose recent submissions are fully covered by the IRT layer retains their own mean, while a student with sparse coverage leans towards the cohort.

Graceful degradation under missing signals The IRT and BKT layers cannot always converge: IRT fails when the response matrix is too sparse and BKT fails when too few observations are available or when all attempts are graded the same way. Rather than setting the values to zero and shrinking the total contribution, the improved model redistributes the unavailable weight back to the behavioural component; this preserves the weight-sum invariant $w_{\text{beh}} + w_{\text{mg}} + w_{\text{da}} = 1$ in every degraded scenario, so configured weights continue to match effective contributions. Table 8 summarises the four cases:

Scenario	w_{beh}	w_{mg}	w_{da}
All signals available	0.45	0.30	0.25
BKT mastery unavailable (or coverage < 50%)	0.75	0.00	0.25
IRT difficulty unavailable (or single distinct value)	0.70	0.30	0.00
Neither signal available	1.00	0.00	0.00

Table 8: Effective weights of the improved struggle model under each combination of available latent-trait signals; redistribution preserves the weight-sum invariant so that configured weights match effective contributions even when the system degrades.

Missing-value treatment for partial mastery The 50% coverage guard handles wholesale unavailability of the mastery signal; the more common case is partial coverage, in which some users have a mastery record and some do not. The improved model fills in missing $\bar{m}_s$ values with the cohort mean rather than zero [?]; using zero would make it so every uncovered student has zero mastery, biasing their mastery gap upwards and over-flagging them. Mean imputation preserves the cohort-level scale and remains neutral on the affected sub-population.

Final shrinkage The combined score $S^{\text{imp}}(s, \sigma)$ is shrunk towards the cohort mean by the same Bayesian estimator used in Section 3.3.1, with the same shrinkage strength $K = 5$. Students with very few submissions are therefore stabilised by the same mechanism in both models; this prevents an unstable mastery gap or difficulty-adjusted estimate on a low- n student from producing a misleading headline score in the dashboard.

3.5 Retrieval-Augmented Generation Feedback Design

This section describes the retrieval-augmented generation (RAG) layer that produces coaching suggestions for the lab assistants and instructors. The architecture adopts a hybrid two-layer retrieval architecture, in which a structured pre-filter narrows the candidate set before a semantic search ranks it (Figure 8). The design grounds LLM output in the session’s own submission history; this mitigates the hallucination risk that has been flagged as a major concern when deploying generative models in educational settings [?, ?].

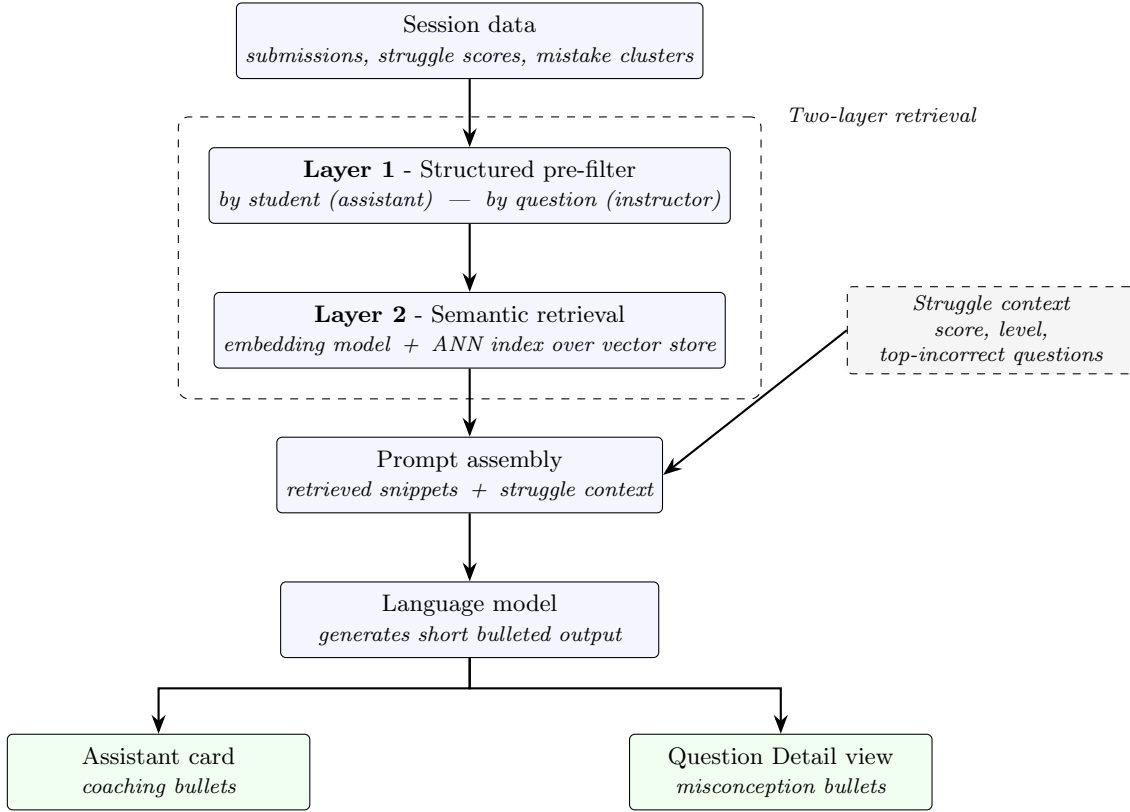


Figure 8: Two-layer retrieval-augmented generation pipeline describing the whole process from session data to final output for both assistants and instructors.

Two-layer retrieval Each layer in the architecture applies a different selection mechanism, and the two together narrow down the search space before generation.

- **Layer 1: Structured Filter** - A query about a single student is restricted to that student's submissions, and a query about a single question is restricted to submissions on that question. The pre-filter operates over the structured attributes already present in the session data, returning only the rows relevant to the query.
- **Layer 2: Semantic Search** - The pre-filtered submissions are ranked by similarity to the query in a vector embedding space [?], with an approximate nearest neighbour index [?] used in place of exact search to keep retrieval within the dashboard's interactive refresh rate. The structured filter is repeated at this layer through metadata constraints so the two layers cannot disagree.

The pre-filter narrows the candidate set to documents that are relevant to the request; the semantic layer then ranks that smaller set by content similarity. The reverse ordering, where the semantic search is first and the structured filter is second, would embed every submission, retrieve a globally similar subset, and then discard most of it, which wastes embedding effort on submissions that were never going to satisfy the structured constraints.

Prompt construction and outputs The retrieved documents are passed to an LLM alongside structured context derived from the rest of the dashboard: the student's struggle score and level, their highest incorrectness questions, and the retrieved submission snippets themselves. The model is instructed to return a short bulleted summary for both assistants and instructors.

Tailoring the pipeline to two audiences The same RAG pipeline serves two distinct audiences with different needs: lab assistants and instructors. On the assistant side, the structured pre-filter narrows to a single student and the output is framed as one-on-one coaching advice. On the instructor side, the pre-filter narrows to a single question and the output is framed as class-wide feedback on common mistakes and misconceptions grounded in the mistake clusters from Section 3.3.5. Sharing the pipeline avoids duplicating the embedding and retrieval infrastructure, while the surface-specific prompts keep each output relevant to its audience.

3.6 Interaction and UI Designs

This section describes how the output of our models defined in Section 3.3.1 and 3.3.3 is presented to instructors in a way that enables quick decision-making during lab sessions. The design is intended to provide an intuitive view.

3.6.1 Design Principles

The user interface is driven by the requirement to expose the model outputs defined in Sections 3.3.1 and 3.3.3 in a form that enables rapid, intuitive interpretation and decision-making. Unlike the previous dashboard we explored, this one will be more intuitive and will be used primarily for computer labs.

The primary objective is to present student struggle and question difficulty as distinct signals. Student struggle indicators are derived from the struggle score $S_t(s, \sigma)$ and question difficulty indicators are derived from the difficulty score $D_t(q)$. These two indicators will prove to be valuable in helping educators improve learning outcomes.

A further objective is to allocate lab assistants to students so they can help promptly. This will send a notification to the lab assistant allocating them to students. Another is avoiding exposure of raw data, which has been mitigated by data normalisation.

Finally, the system focuses on supporting decision-making rather than explanatory analysis. The dashboard is intended to guide instructors towards timely intervention by highlighting students and tasks requiring attention and enabling efficient allocation of lab assistants.

3.6.2 Instructor Views

The dashboard provides a real-time overview of student struggles and question difficulty during the computer labs. The dashboard will allow the instructor to make quick decisions and aid those in need. To illustrate the proposed interface, three Figma mock-ups are presented in Figma 9 - 11. The deployed implementation that follows this design is described in the implementation chapter.

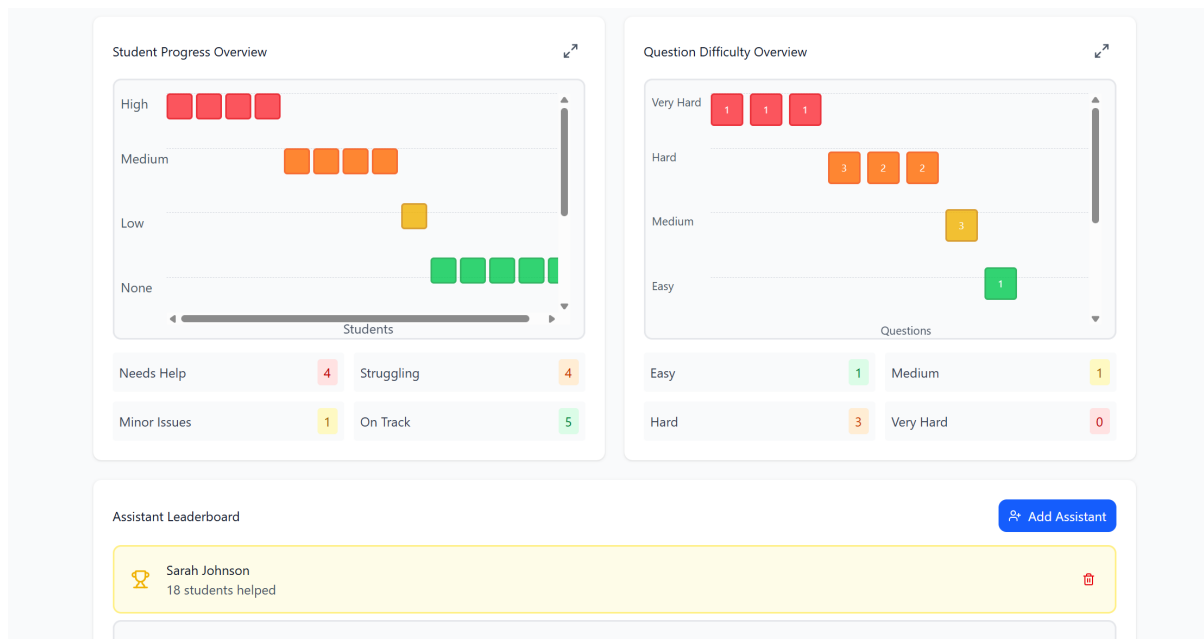


Figure 9: Views based on student struggle and question difficulty

As shown in Figure 9, our dashboard has two key views: the first shows a list of students whose struggles are at certain thresholds, hence the colour; the second shows a list of questions based on difficulty at a threshold, thus the colour. The values used for comparison against the thresholds are derived from our student struggle and question difficulty equations. The view also allows instructors to go over additional information.

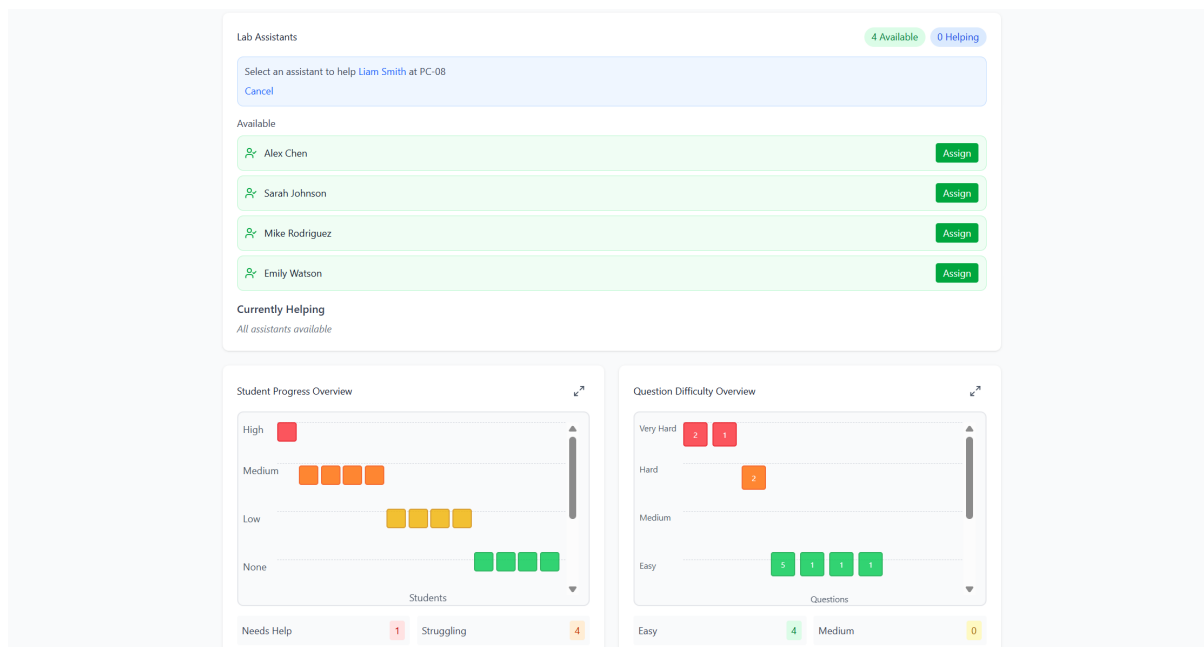


Figure 10: View of allocation of Lab Assistant

In Figure 10, we can select a lab assistant and then press a student to allocate them. This is a simple approach and a very minimal, intuitive design that allows use without training on the system.

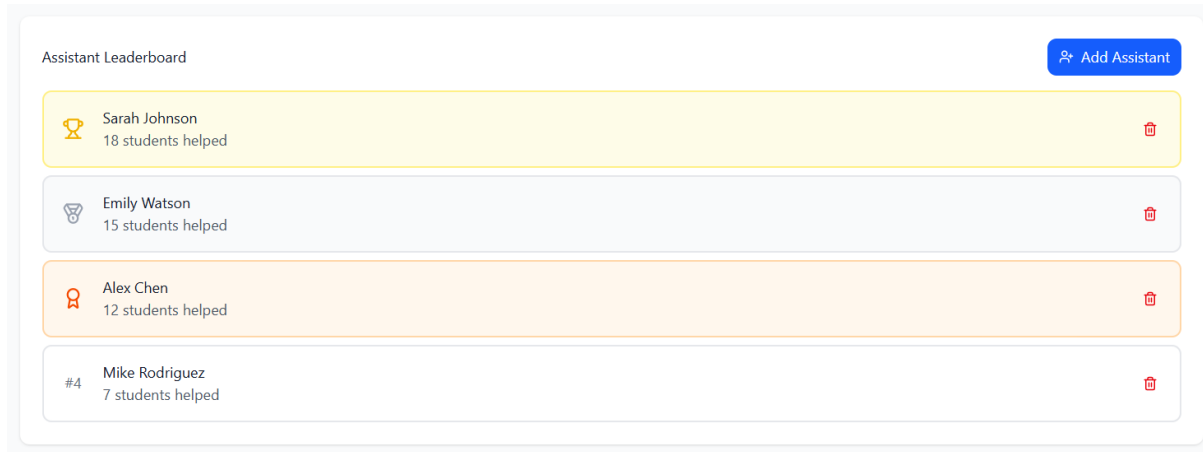


Figure 11: Assistant leaderboard, based on how many students have been helped

In Figure 11, we can see that lab assistants are displayed based on their contributions. By gamifying the helping process, we can incentivise assistants to help more students.

3.6.3 Lab Assistant View

The lab assistant view is the mobile interface designed for assistants moving between students during a live session. Unlike the lab instructor dashboard, which assumes a stationary screen and wide canvas, the assistant view is built for short interactions on a handheld device. The interface presents one of three states at any moment, each scoped to the assistant's current responsibility: joining a session, waiting for an assignment, or helping the student they have been assigned. Figure 12 shows the three states in sequence.

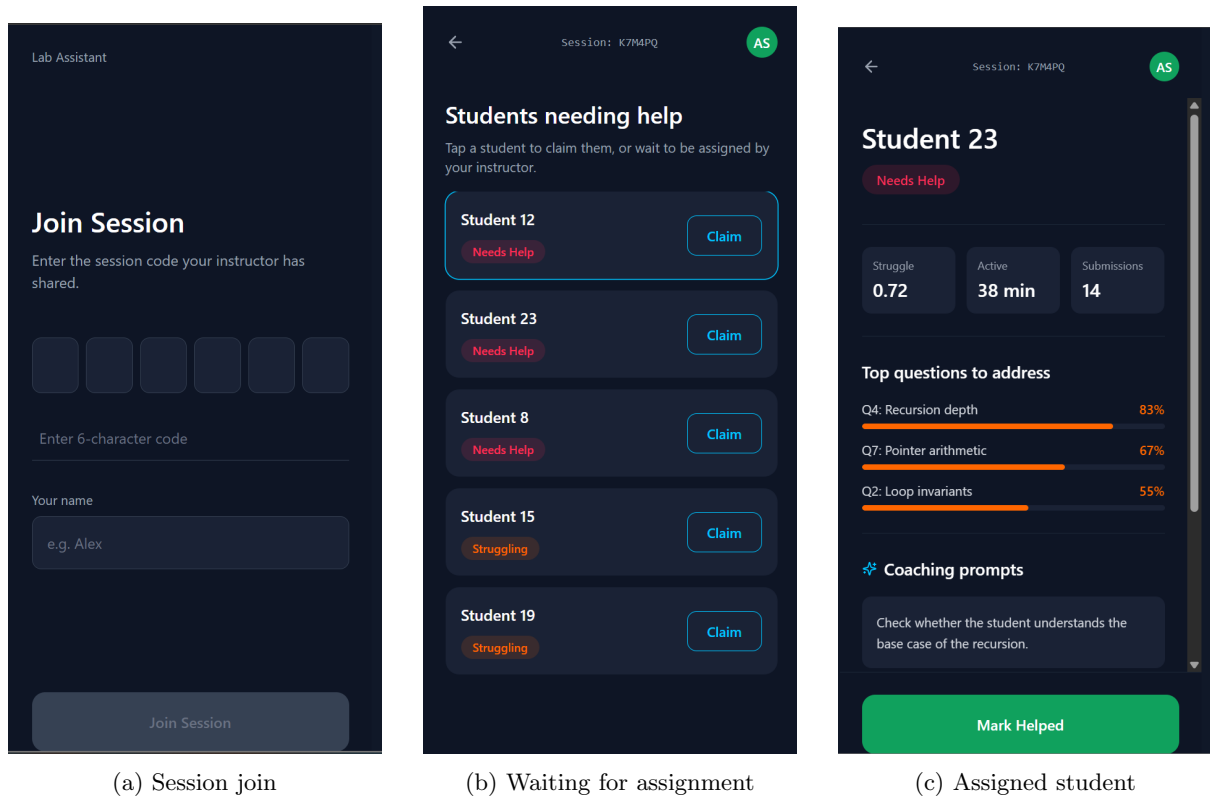


Figure 12: The three states of the lab assistant view: a the session-join screen where the assistant enters the session code and their name; b the waiting state with a priority list of unassigned struggling students; c the assigned-student card showing struggle context, prioritised questions, and coaching prompts from the RAG layer.

On first access (Figure 12a), the assistant enters the session code shown on the instructor dashboard and provides their name. The session code is short enough to be read aloud by the instructor and typed without errors; the name is recorded so subsequent allocation decisions can address assistants individually. Once joined, the assistant is enrolled in the session and remains until they leave, are removed, or the session ends.

Between assignments (Figure 12b), the assistant sees a ranked list of students currently flagged for help, ordered by their struggle score. The assistant may wait for the instructor to allocate them directly, or self-claim a student from the list when the instructor has enabled this. The same list is visible to every joined assistant; once a student is claimed, they are removed from the unassigned pool so two assistants cannot end up at the same desk.

Once a student is assigned (Figure 12c), the view collapses to a single student card carrying the information the assistant needs at the desk: the student’s current struggle context, the questions they are most incorrect on, and a short list of coaching prompts retrieved from the RAG layer described in Section 4.8. A *Mark Helped* action returns the assistant to the unassigned state.

3.6.4 Visual Encoding of Metrics

By deciding on thresholds, we can highlight a student by degree of struggle or highlight a question by degree of difficulty

Degree	On Track	Minor Issues	Struggling	Needs Help
Threshold	$0 \leq S_t(s, \sigma) \leq t_{s1}$	$t_{s1} < S_t(s, \sigma) \leq t_{s2}$	$t_{s2} < S_t(s, \sigma) \leq t_{s3}$	$t_{s3} < S_t(s, \sigma) \leq 1$

Table 9: Table Highlighting how thresholds can be indicated for struggle

Degree	Easy	Medium	Hard	Very Hard
Threshold	$0 \leq D_t(q) \leq t_{s1}$	$t_{s1} < D_t(q) \leq t_{s2}$	$t_{s2} < D_t(q) \leq t_{s3}$	$t_{s3} < D_t(q) \leq 1$

Table 10: Table Highlighting how thresholds can be indicated for difficulty

4 Implementation

4.1 Implementation Overview

The dashboard is deployed in two versions, both are fully featured and runnable in parallel from a single working tree. Version 1 (`code/`) is a Streamlit implementation that consists of two separate apps: one for instructors and one for lab assistants, each running their own Python process. Version 2 (`code2/`) is a single app, FastAPI process that serves both an asynchronous HTTP API at `/api/` and two built React single-page applications (SPAs) for instructors and assistants respectively. V2 builds on V1, with every feature from V1 reimplemented in V2, additional features added on top and performance improvements.

An identical analytical design sits behind both iterations of the dashboard. Data ingestion, scoring, the advanced model layer, RAG feedback and saved sessions are realised in V1 as the `learning_dashboard/` package under `code/`, and in V2 within the `backend/` package under `code2/`. V2 splits V1's monolithic `analytics.py` into per-signal files and adds a thin infrastructure layer (Pydantic schemas, dependency injection, a disk-backed scoring cache) required by the stateless request model. The data pipeline, the runtime-state sharing, and the threshold-classification machinery (the bands and the `classify_score` helper) carry across both versions unchanged. The scoring outputs, however, have developed in several places: V2 fits a 2PL IRT model that also estimates student abilities where V1 fits a Rasch (1PL) model, V2 fits BKT parameters once at FastAPI startup with the `lifespan` handler and holds them in an in-process cache, so the four-parameter model is fit once per process lifetime and reused across stateless requests where V1 leaves the parameters at their configured defaults from `config.py` (unless changed through the Settings sliders), V2's improved struggle model adds an IRT-residual difficulty adjusted path (`expected_correct - observed_correct` using the 2PL ability and discrimination) that V1 has no equivalent for, and V2 weights the baseline struggle signal by per-row measurement confidence where V1 takes the unweighted mean. Both versions share a single runtime-state file, `data/lab_session.json`, coordinated through the `filelock` library under a five-second exclusive lock, so that a session started in V1 becomes visible to V2 within a few seconds and vice versa. The implication is that the analytical contribution is decoupled from the presentation layer, and where V2 extends V1 it does so within the same analytical framework with the same signal categories, the same threshold bands, the same combination structure, whilst refining the numerics that feed into each.

The motivation behind the second iteration of the dashboard is highlighted later on in the section. The first iteration had a Streamlit foundation that was ideal for rapid iteration and early user feedback, but was limiting in terms of performance, scalability and flexibility. The limitation of rerunning the entire script on every interaction meant that the system struggled to maintain a responsive user experience which complicates asynchronous work. The second iteration addresses these limitations by moving the presentation layer to React and the backend to FastAPI whilst keeping the analytical core untouched. The comparison between V1 and V2 is a recurring theme in the implementation section, as it provides a useful lens to explain the rationale behind design decisions and the trade-offs involved.

This chapter is organised top-down so that Section 4.2 describes the tech stack used by both versions. Section 4.3 describes the runtime processes, the shared runtime state and the deferred-actions pattern that underpins V1's reactive interactions. Section 4.4 and 4.5 cover the data pipeline from the lab endpoint to the in-memory dataframe and the live and saved-session lifecycles. Section 4.6 covers the baseline analytics and Section 4.7 covers the advanced modelling layer of measurement confidence, item-response-theory difficulty, Bayesian knowledge-tracing mastery, and the improved struggle model that combines them. Section 4.8 covers the retrieval-augmented feedback layer that produces real-time coaching suggestions for the lab assistant. Section 4.9 and 4.10 walk through the instructor views and the lab-assistant flows respectively. Section 4.11 highlights the implementation problems encountered and the resolutions adopted.

4.2 Technology Stack

The analytical core, written in Python 3.11, is shared between both versions of the dashboard whilst relying on pandas and NumPy for in-memory data manipulation, SciPy for model fitting, ChromaDB and sentence-transformers for the RAG layer, and the OpenAI Python SDK for LLM interactions. The presentation and service layers are where the systems diverge, with V1 using Streamlit for the instructor and assistant dashboards and V2 using FastAPI to serve a React single-page application built with Vite and TypeScript.

Component	Role	Version
Python 3.11	Implementation language for the analytical core and both backends	Both
Streamlit	Instructor and assistant dashboard UI; session-state model	V1
streamlit-autorefresh	Browser-side polling for live updates	V1
Plotly	Interactive charts and leaderboards	V1
FastAPI	HTTP API exposing the analytical core to the React frontend	V2
Uvicorn	ASGI server (with reload) for FastAPI	V2
React 19 + Vite + TypeScript	Single-page application; seven runtime themes	V2
pandas, NumPy	In-memory data frame and vectorised scoring	Both
SciPy	L-BFGS-B optimiser for IRT (Rasch 1PL in V1, 2PL in V2) and BKT MLE fits	Both
scikit-learn	TF-IDF + KMeans for mistake clustering; <code>roc_auc_score</code> for BKT predictive AUC	Both
OpenAI Python SDK	Batched incorrectness scoring (gpt-4o-mini) and RAG generation	Both
sentence-transformers, ChromaDB	Local MiniLM-L6-v2 embeddings + persistent vector store for RAG	Both
filelock	Cross-process write lock for the shared lab-session JSON	Both
requests	HTTP client against the lab data endpoint	Both
cachetools	TTL cache for the FastAPI analytical layer	V2

Table 11: Technologies and libraries used in V1 (Streamlit) and V2 (React + FastAPI)

4.3 System Structure

In this section, we describe the overall structure of the dashboard system, including the runtime processes, the shared runtime state and the deferred-actions pattern. V1's Streamlit foundation means that the instructor and assistant dashboards run in separate Python processes, each with its own memory space and runtime state. V2's FastAPI backend serves both the instructor and assistant React frontends from a single process as seen in Figure 13. All three apps share one runtime-state file (`data/lab_session.json`) for live session coordination.

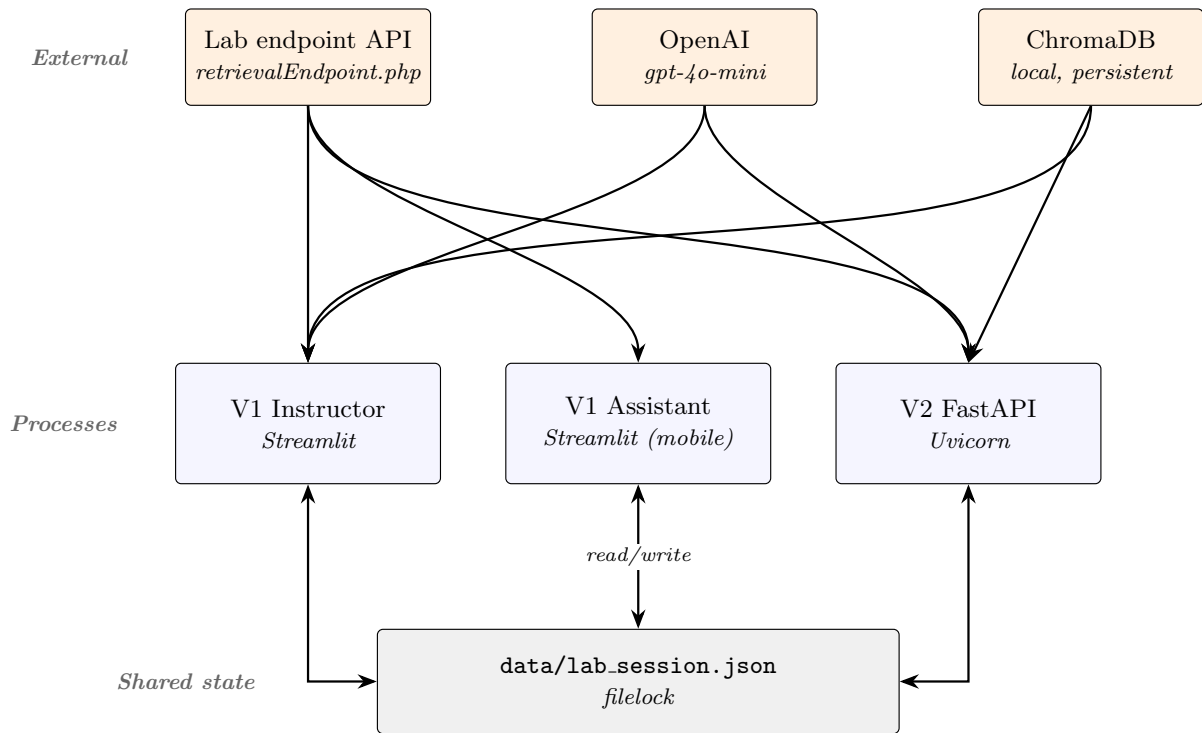


Figure 13: Architecture of the deployed system showing V1 (two Streamlit processes) and V2 (one FastAPI process) sharing the file-locked `data/lab_session.json`.

4.3.1 V1 - Instructor Process (Streamlit)

The V1 instructor process is entered through `code/app.py`, a thin wrapper that calls `main()` entry point in `learning_dashboard.instructor_app`. Streamlit's execution model is single-threaded meaning that simultaneous users each receive their own server-side `st.session_state` keyed by Streamlit's session identifier. The sidebar drives both the filtering (temporal and module filters) and the live-session controls (start, end, the six-character lab code and the joined-assistants roster). View routing is held in the `current_view` session-state key, updated through a callback whenever the user picks from the view selector; the main pane renders one of the seven views described in Section 4.9 based on this value.

4.3.2 V1 - Assistant Process (Streamlit, mobile)

The V1 lab assistant process is entered through `code/lab_app.py`, a thin wrapper around the `main()` entry point in `learning_dashboard.assistant_app`, intended for phone-sized viewports. Assistant identity is persisted across reloads through a URL `?aid=<slug>_<nnnn>` query parameter (for example, `?aid=alice.4827`), set when the assistant first joins and read on every subsequent render. The page auto-refreshes every five seconds via the `st.autorefresh` helper from `streamlit-autorefresh`, independently of the instructor app's refresh cycle. Each render reads the shared lab-session state described in Section 4.3.4 and forwards to one of three views: a join screen when no `aid` is present (Figure 22), an unassigned standby view when an `aid` is present but no student is assigned (Figure 23) or the assigned view when the assistant has been assigned to a student (Figure 24).

4.3.3 V2 - React + FastAPI

V2 further refines V1's architecture by addressing four limitations encountered after V1 reached feature completion. Streamlit's execution model reruns the entire script on every interaction, so long-running operations such as the RAG collection build or the BKT Maximum Likelihood Estimation (MLE) fit block the user-interface thread and force the cold start mitigation described later in Section 4.4 and Section 4.6. There is no native support for asynchronous code, so awaiting an OpenAI or ChromaDB response would block the entire app, which is particularly problematic for the lab assistant's real-time coaching suggestions. The theming model is limited in contrast to the seven-theme palette and accent colour toggle added in V2, and implementing a custom theme switcher in Streamlit would have required a non-trivial amount of custom CSS and JavaScript injection. Multi-page navigation in Streamlit is route-based with a full-page reload, so V1 works around this by holding the active view in session state and switching the rendered pane on each rerun.

V2 consolidates the two V1 processes into a single FastAPI process entered through `code2/backend/main.py`. The same process exposes the analytical core at `/api/*` and serves two HTML entry points from the built `frontend/dist/` directory: `index.html` is the instructor counterpart to V1's `code/app.py`, and `mobile.html` is the lab-assistant counterpart to V1's `code/lab_app.py`. The two entry points share the same React bundle but have different route components and separate routing trees. View-level descriptions for both flows live in Section 4.9 and Section 4.10, with the V1 versus V2 comparison captured in Table 14.

Eleven routers under `code2/backend/routers/` adapt the analytical core to HTTP: `live`, `student`, `question`, `analysis`, `sessions`, `settings`, `models_cmp`, `lab`, `rag`, `meta` and `cf`. The `mobile.html` bundle calls `/api/lab/*`; the `index.html` bundle calls the others. The frontend is Vite, React and TypeScript with nine top-level views, eight of them are on the instructor side and one for the lab assistant; the eight instructor views include all the ones V1 carries with the addition of a `SessionProgression` view that replays struggle evolution across the session, an interaction pattern V1's rerun model cannot express. V2 also adds a seven-theme runtime palette (paper, newsprint, solar, scifi, blueprint, matrix, cyberpunk) swappable from the settings view through React Context and CSS custom properties; the accent colour is picked separately and persisted to `localStorage`. A FastAPI `lifespan` handler runs at startup and prewarms the data frame, incorrectness scores, the struggle and difficulty leaderboards, the BKT and IRT fits and the ChromaDB embedding collection so that the first user request is not subject to cold-start latency. Figure 14 shows the V2 In-Class view in the scifi theme.



Figure 14: Screenshot of V2 (React + FastAPI) showing the In-Class view in the sci-fi theme; the analytical layer is reused from V1 with the divergences noted in Section 4.1.

V2 carries the same analytical design as V1 with the same signal categories, the same threshold bands, the same pipeline and same retrieval procedure. The V1 modules live inside the `learning_dashboard/` package under `code/` and the V2 modules live inside the `backend/` package under `code2/`, alongside `main.py` and the routers (Table 12). The scoring outputs evolve in four places between the two versions, as detailed in Section 4.1: V2's IRT, BKT, improved struggle and baseline struggle each carry refinements over V1. The threshold classification is unchanged, allowing the V1 and V2 visual encodings to remain consistent.

Concern	V1 (code/learning_dashboard/)	V2 (code2/backend/)
Entry point(s)	<code>instructor_app.py</code> , <code>assistant_app.py</code>	<code>main.py</code> (FastAPI)
Data ingestion	<code>data_loader.py</code>	<code>data_loader.py</code>
Analytical core	<code>analytics.py</code> (monolithic)	<code>analytics.py</code> plus per-signal split: <code>incorrectness.py</code> , <code>struggle.py</code> , <code>difficulty.py</code> , <code>clustering.py</code> , <code>collab.py</code>
Shared session state	<code>lab_state.py</code>	<code>lab_state.py</code>
Configuration	<code>config.py</code> , <code>paths.py</code>	<code>config.py</code> , <code>paths.py</code> , <code>runtime_config.py</code>
Academic calendar	<code>academic_calendar.py</code>	<code>academic_calendar.py</code>
RAG	<code>rag.py</code>	<code>rag.py</code>
Class tagging	<code>lab_classes.py</code>	<code>lab_classes.py</code>
Audio cues	<code>sound.py</code>	(browser-side in V2)
Advanced models	<code>models/</code> (irt, bkt, improved_struggle, measurement)	<code>models/</code> (same four files)
UI / presentation	<code>ui/</code> (views, components, theme) - Streamlit	<code>frontend/</code> - separate React + Vite tree
HTTP layer		<code>routers/</code> (11: live, student, question, analysis, sessions, settings, models_cmp, lab, rag, meta, cf)
Request schemas		<code>schemas.py</code>
Dependency injection		<code>deps.py</code>
In-process cache		<code>cache.py</code>
Demo data		<code>demo_data.py</code>

Table 12: Module organisation of the analytical core in V1 (left column) and V2 (right column)

4.3.4 Shared Runtime State

All three apps share a single runtime-state file at `data/lab_session.json`, accessed through the `filelock` library under a five-second exclusive lock. Writes are atomic, meaning that the new state is written to a `.tmp` file first and then renamed via `os.replace`, so a reader never observes a partial JSON file (Figure 15). The file holds the active session metadata, the roster of joined assistants keyed by identifier, and an assignment map from student to assistant with a `helping` or `helped` status. A change made by any process becomes visible to the others within one refresh cycle.

```

38 def _lock() -> FileLock:
39     lock_file = paths.lab_session_lock_path()
40     lock_file.parent.mkdir(parents=True, exist_ok=True)
41     return FileLock(str(lock_file), timeout=_LOCK_TIMEOUT_SECONDS)

131 def _read_state_unlocked() -> dict[str, Any]:
132     state_file = paths.lab_session_path()
133     if not state_file.exists():
134         return _default_state()
135     try:
136         raw = json.loads(state_file.read_text(encoding="utf-8"))
137     except (OSError, json.JSONDecodeError):
138         return _default_state()
139     return _normalize_state(raw)

142 def _write_state_unlocked(state: dict[str, Any]) -> None:
143     state_file = paths.lab_session_path()
144     state_file.parent.mkdir(parents=True, exist_ok=True)
145     normalised = _normalise_state(state)
146     tmp_file = state_file.with_suffix(".tmp")
147     tmp_file.write_text(json.dumps(normalised, indent=2), encoding="utf-8")
148     tmp_file.replace(state_file)

```

Figure 15: File-locked read/write pattern for the shared lab-session state

4.3.5 Deferred-Actions Pattern

Streamlit forbids modifying `st.session_state` after a widget has been instantiated in the same run; doing so raises a `StreamlitAPIException`. V1 deals with this constraint through a deferred-actions pattern where callbacks and post-widget click handlers do not modify the session state directly, but instead set a `pending.*` flag on the session state describing the change to be made. The next rerun reads those flags inside `main()`, before the sidebar widgets that consume these flags are constructed, applies the changes and clears the flag. This pattern avoids the need for extra threads or queues for state coordination, and keeps each transition atomic in the sense that one rerun applies one change (Figure 16). V2 does not need this pattern, as React and FastAPI do not impose the same widget-instantiation rule.

(a) Button callback inside the Previous Sessions view sets the `pending_*` flag and reruns:

```

927 if st.button("Load", key=f"load_session_{session_id}"):
928     st.session_state["pending_session_load_record"] = record
929     st.session_state["pending_delete_session_id"] = None
930     st.rerun()

```

(b) Inside `main()` on the next rerun, *before* the sidebar widgets bound to these keys are constructed: read, apply, clear.

```

763 pending_record = st.session_state.get("pending_session_load_record")
764 if isinstance(pending_record, dict):
765     available_modules = (
766         ["All Modules"] + sorted(df["module"].unique().tolist())
767         if not df.empty
768         else ["All Modules"]
769     )
770     data_loader.apply_saved_session_to_state(
771         pending_record,
772         available_modules=available_modules,
773     )
774     st.session_state["pending_session_load_record"] = None
775     st.session_state["_nav_loading"] = True
776     st.rerun()

```

Figure 16: The deferred-actions pattern used to mutate session state without violating Streamlit's widget-instantiation rule. Block (a) is the click handler, block (b) is the matching apply-and-clear.

4.4 Data Pipeline

The data pipeline is a five-step path from the lab endpoint to the in-memory dataframe which gets consumed by the scoring layer: endpoint polling, detection of the JSON-vs-XML payload, parse, normalise, cache. Both V1 and V2 use the same modules for the first four steps and diverge only at the caching layer, where V1 relies on Streamlit's `@st.cache_data` decorator and an in-process dictionary whilst V2 layers `cachetools` time-to-live (TTL) caches over a disk-persisted incorrectness cache that survives process restarts (Figure 17).

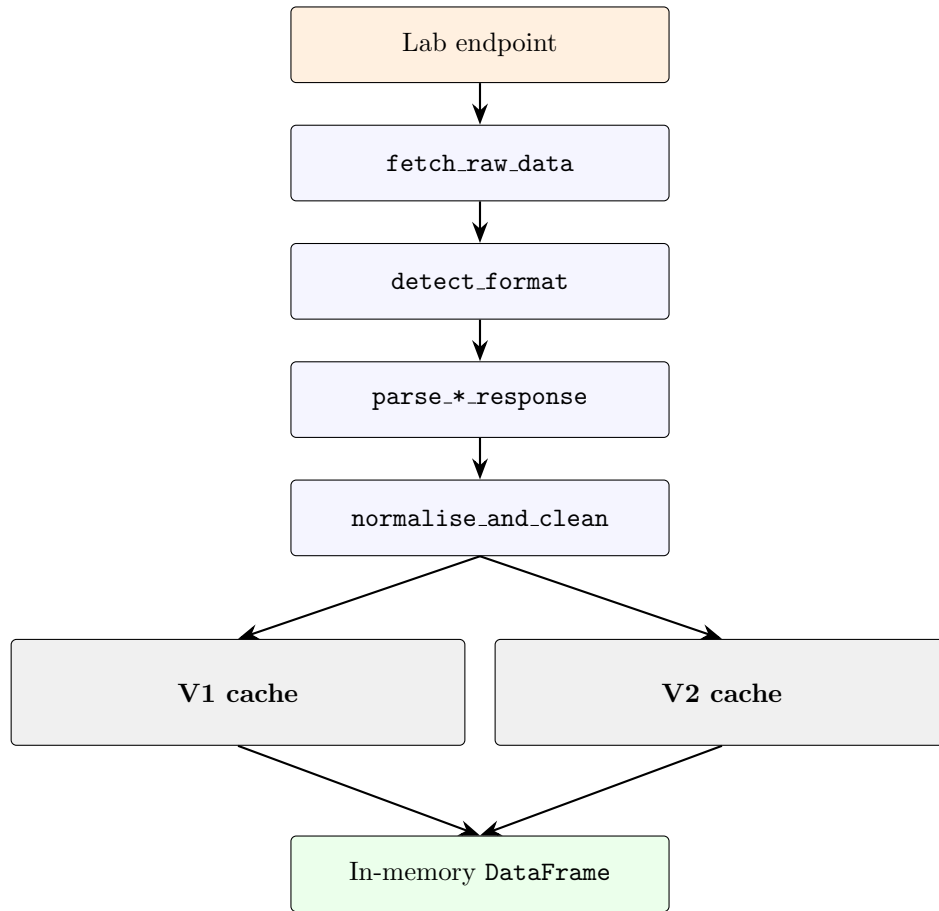


Figure 17: Data pipeline from the lab endpoint to the in-memory DataFrame. The fetch, format-detection, parse and normalise stages are shared between V1 and V2; the cache stage forks into V1’s two-tier in-memory cache and V2’s layered `TTLCache` stack plus disk-persisted incorrectness cache.

Endpoint retrieval and parsing The endpoint is Loughborough’s existing PHP retrieval URL, named in `config.py` as `API_URL`; the same URL is used by both versions. The HTTP fetch is a synchronous `requests.get()` call with a 30-second timeout (`API_TIMEOUT = 30`). Under normal operation the endpoint returns newline-delimited JSON, with a pure-XML payload retained as a fallback for legacy endpoint behaviour; format detection is a single-attempt JSON parse on the first non-empty line. Each JSON record carries the outer fields `module`, `question`, `session` and `user`, plus an inner `xml` string. The inner XML carries one or more `<submission>` elements, each with a `timestamp` attribute, an `<srep>` element (the student’s answer), and a `<feedback>` element (the AI feedback text consumed downstream by the incorrectness scorer); the pipeline emits one DataFrame row per submission element. Embedded XML is parsed with the standard-library `xml.etree.ElementTree` (Figure 18).

```

59 xml_field = obj.get("xml", "")
60 if xml_field and isinstance(xml_field, str) and xml_field.strip():
61     try:
62         root = ET.fromstring(f"<root>{xml_field}</root>")
63         submissions = root.findall("./submission")
64         if submissions:
65             for sub in submissions:
66                 record = base.copy()
67                 record["timestamp"] = sub.get(
68                     "timestamp", obj.get("timestamp", "")
69                 )
70                 srep = sub.find("srep")
71                 record["student_answer"] = (
72                     srep.text if srep is not None and srep.text else ""
73                 )
74                 feedback_el = sub.find("feedback")
75                 record["ai_feedback"] = (
76                     feedback_el.text
77                     if feedback_el is not None and feedback_el.text
78                     else ""
79                 )
80                 records.append(record)

```

Figure 18: The embedded-XML extraction inside `parse_json_response()`: the per-record `xml` string is wrapped in a synthetic `<root>` so `xml.etree.ElementTree` can iterate the contained `<submission>` elements, each producing one `DataFrame` row.

Data normalisation and structuring All records collapse into a single pandas `DataFrame` with seven columns: `module`, `question`, `timestamp`, `student_answer`, `ai_feedback`, `session` and `user`. An eighth column, `academic_period`, is added by `academic_calendar.add_academic_period_column()` and maps each row's timestamp to a Loughborough 2025/26 academic-week label which is consumed by the Data Analysis view in Section 4.9. The excluded modules filter (`EXCLUDED_MODULES`) drops test and admin modules before any scoring and modelling steps. Timestamps are parsed with `pandas.to_datetime(..., errors="coerce")`, rows with unparseable timestamps are dropped and the `DataFrame` is sorted by timestamp.

Caching V1 caches at two layers. `fetch_raw_data()` is wrapped in `@st.cache_data(ttl=CACHE.TTL)` with `CACHE.TTL = 10` seconds, so successive sidebar reruns within the same refresh cycle do not re-hit the endpoint. A session-state shortcut in `load_data()` hashes the raw response and skips the parse and normalise steps when the hash is unchanged from the previous render. The incorrectness scorer in Section 4.6 maintains its own in-process dictionary `_incorrectness_cache` in `analytics.py`, which is keyed by feedback text; it is local to the process and does not survive a restart. The Streamlit auto-refresh widget runs on a separate cadence (`AUTO_REFRESH_INTERVAL_DEFAULT = 300` seconds, configurable from five seconds up to five minutes), the 10-second `CACHE.TTL` deduplicates within one refresh cycle, whilst the chosen auto-refresh interval controls how often a fresh cycle happens.

V2 retains the same in-process scorer cache but enhances the storage in three places. The raw-`DataFrame` cache becomes a `cachetools.TTLCache` with `maxsize=1` and a 60-second TTL (`CACHE.TTL = 60` in `code2/backend/config.py`), since FastAPI's stateless request model would otherwise rebuild the `DataFrame` on every endpoint hit. Above that, four additional `TTLCache` layers sit one level higher in the call stack: `_struggle_cache` and `_difficulty_cache` at 15 minutes, `_improved_cache` and `_irt_difficulty_cache` at 10 minutes, each caching the scoring outputs per session identifier. On top of both, V2 also adds a disk-persisted incorrectness cache at `data/incorrectness_cache.json`, written through `code2/backend/incorrectness`. A FastAPI `lifespan` handler runs at startup, loads the `DataFrame`, computes the warm ratio (the fraction of pending feedback strings already on disk) and if the warm ratio is at least 0.95 it short-circuits

the scoring pass entirely, otherwise it batches the missing feedback strings to OpenAI and updates the cache file (Figure 19). The same handler then fits the BKT and IRT models once and stashes their parameters in memory for reuse across requests.

```

94 feedbacks = df["ai_feedback"].astype(str).str.strip()
95 unique_fbs = [t for t in feedbacks.unique() if t]
96 if unique_fbs:
97     cached_count = sum(
98         1 for t in unique_fbs
99         if t in incorrectness._incorrectness_cache
100     )
101     warm_ratio = cached_count / len(unique_fbs)
102 else:
103     warm_ratio = 1.0
104 if warm_ratio >= 0.95:
105     df["incorrectness"] = incorrectness.compute_incorrectness_column(
106         df, score_new=False
107     )
108 else:
109     df["incorrectness"] = incorrectness.compute_incorrectness_column(
110         df, max_new_scores=0
111     )

```

Figure 19: V2 startup warm-ratio short-circuit. The fraction of unique feedback strings already present in the disk-persisted cache is computed; at $\geq 95\%$ the lookup-only path runs (no OpenAI calls), otherwise the full scoring pass runs once.

The net effect is that V2’s cold-start cost is paid once per process restart rather than once per Streamlit rerun, converting the rerun-time scoring cost that V1’s per-run cap (`SCORING_PER_RUN_CAP = 500`) is built to absorb into a one-shot startup cost paid before any user request reaches the routers. V2 therefore runs significantly faster than the V1 and give a greater user experience (Table 13).

Cache	Scope	TTL	Persists	Version
<code>@st.cache_data</code> on <code>fetch_raw_data</code>	Raw endpoint response	10 s	In-memory	V1
<code>_df_hash</code> session-state skip	Parsed DataFrame	Per-render	In-memory	V1
<code>_incorrectness_cache</code> dict	Feedback $\rightarrow$ score	Process	In-memory	V1
<code>_df_cache</code>	Raw DataFrame	60 s	In-memory	V2
<code>_struggle_cache</code> , <code>_difficulty_cache</code>	Per-session scoring output	15 min	In-memory	V2
<code>_improved_cache</code> , <code>_irt_difficulty_cache</code>	Per-session advanced model	10 min	In-memory	V2
<code>incorrectness_cache.json</code>	Feedback $\rightarrow$ score	—	On disk	V2

Table 13: Caches in the data pipeline across V1 (Streamlit) and V2 (FastAPI).

4.5 Session Management

A session in this system is a bounded window of submissions used for the in-class workflow, which is recorded so it can be revisited later on. The live session lives in `data/lab_session.json` (covered in Section 4.3.4); once ended, the record is appended to `data/saved_sessions.json` for review later in the Previous Sessions view.

The session lifecycle is coordinated through the file-locked state from Section 4.3.4, with every mutation atomic under the write lock.

1. The instructor clicks “Start Session” in the sidebar, which calls `lab_state.start_lab_session()`; this generates a six-character code with `generate_session_code()` and writes the session metadata (`session_code`, `session_active=True`, `session_start`, `class_id`, `class_label`) under the file-locked write helper.
2. Assistants enter the code and a display name on the lab assistant app, which calls `join_session(session_code, name)` and either mints a new `assistant_id` or reuses an existing one if a matching display name is already in the roster, then appends them to the roster.
3. Assignment is bidirectional: the dashboard calls `assign_student(student_id, assistant_id)` or, if the lab instructor has enabled self-allocation, the assistant calls `self_claim_student(student_id, assistant_id)` from the lab app; both adjust the same assignment map under the write lock, and completion is signalled by `mark_student_helped(student_id)`, which stamps `status="helped"` and `helped_at`.
4. Ending the session calls `end_lab_session()`, which sets `session_active=False`, clears the `assignments` map and nulls each assistant’s `assigned_student`; the session is then optionally persisted as a saved record.

The session code is drawn uniformly from `string.ascii_uppercase + string.digits`, giving $\log_2(36^6) \approx 31$ bits of entropy, which is more than sufficient for the expected scale of use and allows for a short and human-friendly code. During a live session the instructor sidebar shows the join code on a card with the elapsed time in `HH:MM:SS` format, the class label and an “End Session” button (Figure 20).

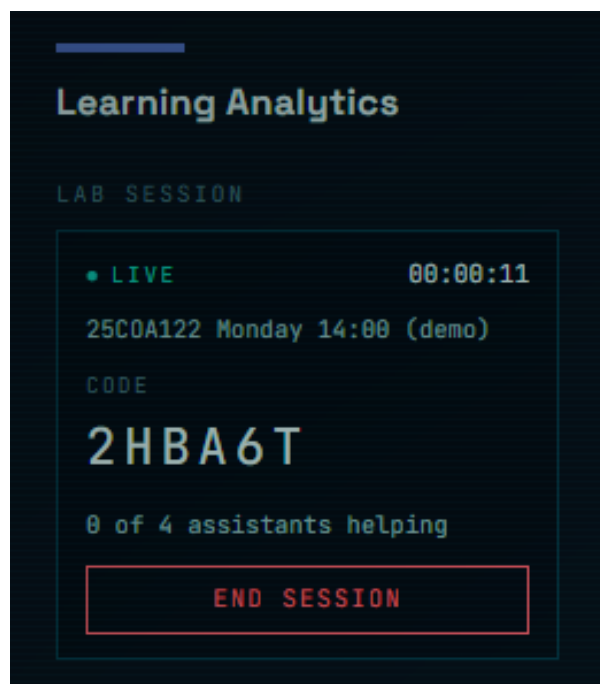


Figure 20: Instructor sidebar during a live session, showing the join code, the elapsed-time timer and the End Session button.

Each ended session is appended to `data/saved_sessions.json` through `save_session_record(record)`; each record carries an `id` (uuid), a `display_name`, `start_time`, `end_time`, `duration.seconds`, `class_id`, `class_label` and a nested `context` dict (Figure 21). The `context` dict carries only the UI filter state; the assistant roster and assignment map are not persisted, since they live in the live-state file

`data/lab_session.json` which is cleared on `end_lab_session()`. This is a deliberate scope split: saved sessions are for retrospective analysis of submissions in a time window and not for replaying assistant allocations. CRUD operations live in `data_loader.py` in both versions and persist through `_write_saved_sessions_payload()`, which uses the same atomic `.tmp` $\rightarrow$ `path.replace(...)` pattern as the live-state writer (Figure 15). In V1, `apply_saved_session_to_state(record, available_modules)` restores the saved filter state to `st.session_state` through the deferred-actions `pending_session_load_record` flag covered in Section 4.3, the next rerun reads the flag, applies the filter state and routes back to the In-Class view at that historical window. V2 has no equivalent server-side function; the React Previous Sessions view re-applies the filter state client-side from the `GET /api/s/{id}` response, sidestepping Streamlit's widget-instantiation rule entirely.

```

1 {
2   "id": "5e3a-...-9c2f",
3   "name": "Week 18 - 25COA122 Monday lab",
4   "class_id": "25COA122-mon-14:00",
5   "class_label": "25COA122 - Monday 14:00",
6   "start_time": "2026-02-09T14:01:23Z",
7   "end_time": "2026-02-09T15:55:12Z",
8   "duration_seconds": 6829,
9   "context": {
10    "dashboard_view": "in_class",
11    "secondary_module_filter": "25COA122",
12    "time_filter_preset": "Custom",
13    "time_date_range": ["2026-02-09", "2026-02-09"],
14    "time_start": "14:00",
15    "time_end": "16:00"
16  }
17 }

```

Figure 21: An example record from `data/saved_sessions.json`. The context dict holds only the UI filter state.

4.6 Analytics Implementation

4.6.1 Incorrectness Scoring

4.6.2 Baseline Student Struggle Model

4.6.3 Baseline Question Difficulty Model

4.6.4 Collaborative Filtering

4.6.5 Mistake Clustering

4.7 Advanced Model Implementation

4.7.1 Measurement Confidence

4.7.2 Item Response Theory (IRT) Difficulty

4.7.3 Bayesian Knowledge Tracing (BKT) Mastery

4.7.4 Improved Struggle Model

4.8 RAG Suggested Feedback

4.8.1 Hybrid Two-Layer Architecture

4.8.2 Embedding Model (all-MiniLM-L6-v2)

4.8.3 Generation and Prompting

4.8.4 Suggestion Caching

4.8.5 Graceful Degradation and Privacy

4.9 Instructor System Views

Table placeholder: V1 vs V2 instructor view parity

Row-per-view comparison of the seven V1 instructor views and the eight V2 instructor views, noting parity, V2-only additions (e.g. SessionProgression) and any V1 features deferred in V2.

Replace this box with the populated tabular environment when §4.9 is written.

Table 14: Parity comparison of instructor views between V1 (Streamlit) and V2 (React + FastAPI).

4.9.1 In-Class View

4.9.2 Student Detail View

4.9.3 Question Detail View

4.9.4 Data Analysis View

4.9.5 Model Comparison View

4.9.6 Settings View

4.9.7 Previous Sessions View

4.10 Lab Assistant System

4.10.1 Session Join Flow

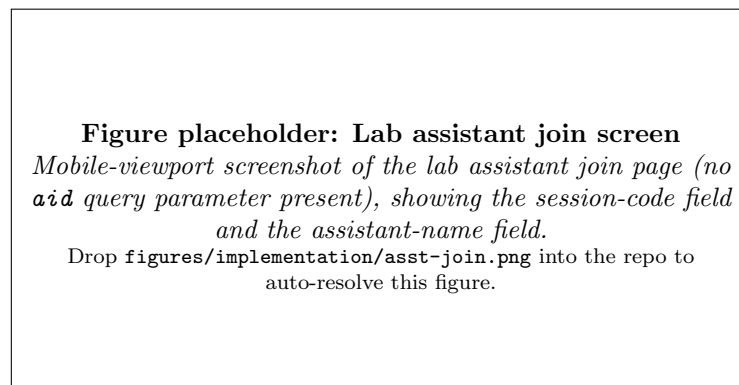


Figure 22: Lab assistant join screen, entered when no `aid` query parameter is bound to the URL.

4.10.2 Waiting and Assignment States

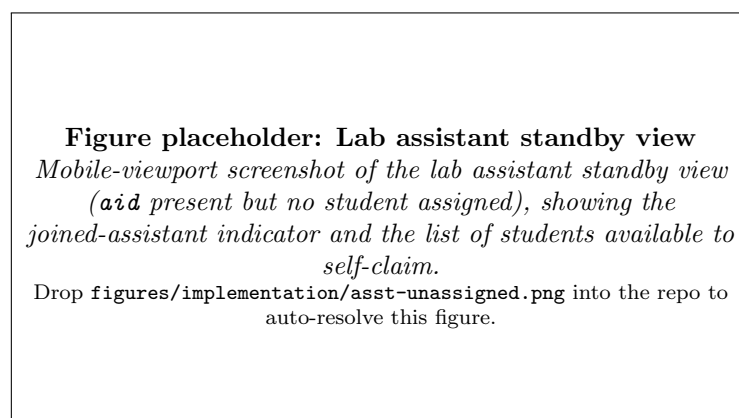


Figure 23: Lab assistant standby view shown once the assistant has joined but before any student is assigned.

4.10.3 Live Assistant Allocation and Mark-Helped

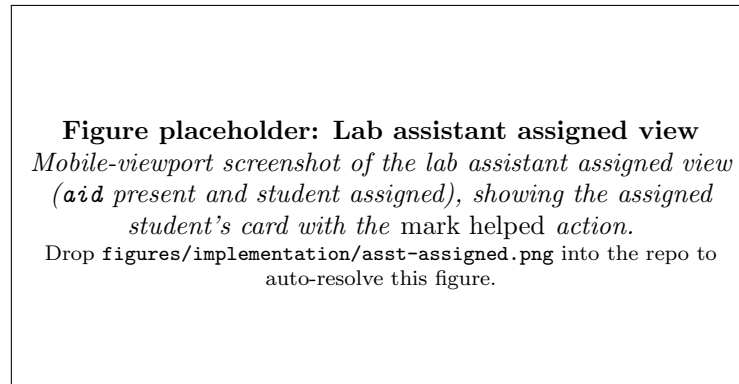


Figure 24: Lab assistant assigned view, showing the student card and *mark helped* action used to close the assignment.

4.11 Problems Encountered and Solutions

5 Evaluation

5.1 Evaluation Design

5.1.1 Scope of Evaluation

5.1.2 Evidence Sources

5.1.3 Retrospective Evaluation Protocol

5.1.4 Limitations of Evaluation

5.2 Functional Testing

5.2.1 Data Ingestion and Session Handling

5.2.2 Analytics and Model Behaviour

5.2.3 Instructor Dashboard Features

5.2.4 Lab Assistant Workflow

5.3 Non-Functional Testing

5.3.1 Performance and Refresh Behaviour

5.3.2 Usability and Interpretability

5.3.3 Robustness and Failure Handling

5.4 Results

5.4.1 Baseline System Results

5.4.2 Advanced Model Comparison

5.4.3 Observed Strengths and Weaknesses

5.5 Discussion

5.5.1 Model Disagreement

5.5.2 Limitations

6 Conclusion

6.1 Summary of the Project

6.2 Key Findings Contributions

6.3 Future Work

A Code Snippets

B User Interface Screenshots

C Detailed Test Results

D Themes and References

Theme	References
Learning Analytics in Higher Education	[?, ?, ?, ?, ?, ?]
Dashboards for Learning Analytics	[?, ?, ?, ?]
Real-Time Data Analytics and Visualisation	[?, ?, ?]
Modelling Student Struggle	[?, ?, ?, ?, ?, ?, ?, ?, ?]
Modelling Question Difficulty	[?, ?, ?, ?]
Collaborative Filtering	[?, ?, ?, ?]
Learning Management System	[?]
Instructor Facing Dashboards	[?, ?]
Early Warning and At Risk Student Systems	[?, ?, ?, ?, ?]

Table 15: Table highlighted the themes covered and the references used in them

E Formulae

F Formulae Derivation

References

- [1] A. Nguyen, L. Gardner, and D. Sheridan, "Data analytics in higher education: An integrated view," *Journal of Information Systems Education*, vol. 31, pp. 61–71, 2020.
- [2] W. Contributors, "Learning analytics," May 2019.
- [3] G. Siemens, "Learning Analytics: The Emergence of a Discipline," *American Behavioral Scientist*, vol. 57, pp. 1380–1400, Oct. 2013.
- [4] D. Clow, "An overview of learning analytics," *Teaching in Higher Education*, vol. 18, pp. 683–695, Aug. 2013.
- [5] C. Vieira, P. Parsons, and V. Byrd, "Visual learning analytics of educational data: A systematic literature review and research agenda," *Computers & Education*, vol. 122, pp. 119–135, July 2018.
- [6] C. Romero and S. Ventura, "Educational data mining and learning analytics: An updated survey," *WIREs Data Mining and Knowledge Discovery*, vol. 10, p. e1355, May 2020.
- [7] A. Gelan, G. Fastré, M. Verjans, N. Martin, G. Janssenswillen, M. Creemers, J. Lieben, B. Depaire, and M. Thomas, "Affordances and limitations of learning analytics for computer-assisted language learning: A case study of the VITAL project," *Computer Assisted Language Learning*, vol. 31, pp. 294–319, Jan. 2018.
- [8] A. Wilson, C. Watson, T. L. Thompson, V. Drew, and S. Doyle, "Learning analytics: Challenges and limitations," *Teaching in Higher Education*, vol. 22, pp. 991–1007, May 2017.
- [9] D. Tzimas and S. Demetriadis, "Ethical issues in learning analytics: A review of the field," *Educational Technology Research and Development*, vol. 69, pp. 1101–1133, Mar. 2021.
- [10] J. Klerkx, K. Verbert, and E. Duval, "Learning analytics dashboards," *Handbook of Learning Analytics*, pp. 143–150, May 2017.
- [11] K. Verbert, X. Ochoa, R. De Croon, R. A. Dourado, and T. De Laet, "Learning analytics dashboards," *Proceedings of the Tenth International Conference on Learning Analytics & Knowledge*, 2013.
- [12] S. Govaerts, K. Leuven, E. Duval, and K. Verbert, "The student activity meter for awareness and self-reflection," May 2012.
- [13] J. Ahn, F. Campos, M. Hays, and D. Digiaco, "The journal of learning analytics works under a creative commons license," vol. 6, pp. 70–85, 2019.
- [14] B. Marr, "What is real-time data analytics (and why it's so important)?," July 2021.
- [15] Mustafa Ayobami Raji, Hameedat Bukola Olodo, Timothy Tolulope Oke, Wilhelmina Afua Addy, Onyeka Chrisanctus Ofodile, and Adedoyin Tolulope Oyewole, "Real-time data analytics in retail: A review of USA and global practices," *GSC Advanced Research and Reviews*, vol. 18, pp. 059–065, Mar. 2024.
- [16] M. Montebello, "Assisting education through real-time learner analytics," 2018.
- [17] Y. Dong, S. Marwan, P. Shabrina, T. Price, and T. Barnes, "Using student trace logs to determine meaningful progress and struggle during programming problem solving."
- [18] P. Or, "Exploring novice programmers' testing behavior: A first step to define coding struggle citation in BibTeX format exploring novice programmer testing behavior: A first step to define coding struggle," 2024.

- [19] A. Estey, H. Keuning, and Y. Coady, “Automatically classifying students in need of support by detecting changes in programming behaviour,” Mar. 2017.
- [20] C. Piech, M. Sahami, D. Koller, S. Cooper, and P. Blikstein, “Modeling how students learn to program.”
- [21] C. Koutchme, N. Dainese, and A. Hellas, “Using program repair as a proxy for language models’ feedback ability in programming education.”
- [22] G. Pitts, A. Hridi, and A.-B. Lekshmi-Narayanan, “A survey of LLM-based applications in programming education: Balancing automation and human oversight.”
- [23] M. Khajah, “Supercharging BKT with multidimensional generalizable IRT and skill discovery.”
- [24] D. Kim, S. Kim, and Y. Jo, “Knowledge tracing in programming education integrating students’ questions.”
- [25] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Modelling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1995.
- [26] M. V. Yudelson, K. R. Koedinger, and G. J. Gordon, “Individualized Bayesian Knowledge Tracing Models,” in *Artificial Intelligence in Education* (D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, H. C. Lane, K. Yacef, J. Mostow, and P. Pavlik, eds.), vol. 7926, pp. 171–180, Berlin, Heidelberg: Springer Berlin Heidelberg, 2013.
- [27] C. Piech, J. Spencer, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein, “Deep Knowledge Tracing,” 2015.
- [28] M. Khajah, R. V. Lindsey, and M. C. Mozer, “How deep is knowledge tracing?,” 2016.
- [29] J. Dannath, A. Deriyeva, and B. Paaßen, “Evaluating task-level struggle detection methods in intelligent tutoring systems for programming,”
- [30] G. Rasch, *Probabilistic Models for Some Intelligence and Attainment Tests*. Studies in Mathematical Psychology, University of Chicago Press, 1960.
- [31] F. Lord and M. Novick, *Statistical Theories of Mental Test Scores*. Addison-Wesley Series in Behavioral Sciences, Emerald Publishing Limited, 1968.
- [32] F. Baucks, R. Schmucker, and L. Wiskott, “Gaining insights into course difficulty variations using item response theory,” Mar. 2024.
- [33] M. Pankiewicz, “Measuring task difficulty for online learning environments where multiple attempts are allowed -the Elo rating algorithm approach.”
- [34] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, “GroupLens: An open architecture for collaborative filtering of netnews,” in *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work - CSCW '94*, (Chapel Hill, North Carolina, United States), pp. 175–186, ACM Press, 1994.
- [35] J. L. Herlocker, J. A. Konstan, A. Borchers, and J. Riedl, “An algorithmic framework for performing collaborative filtering,” in *Proceedings of the 22nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, (Berkeley California USA), pp. 230–237, ACM, Aug. 1999.
- [36] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” *Communications of the ACM*, vol. 18, pp. 613–620, Nov. 1975.

- [37] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 1 ed., 2006.
- [38] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” 1967.
- [39] D. Arthur and S. Vassilvitskii, “K-means++: The Advantages of Careful Seeding,” 2007.
- [40] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, Nov. 1987.
- [41] Z. Wang, J. Shang, and R. Zhong, “Goal-Driven Explainable Clustering via Language Descriptions,” 2023.
- [42] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” 2020.
- [43] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier, S. Krusche, G. Kutyniok, T. Michaeli, C. Nerdel, J. Pfeffer, O. Poquet, M. Sailer, A. Schmidt, T. Seidel, M. Stadler, J. Weller, J. Kuhn, and G. Kasneci, “ChatGPT for good? On opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, p. 102274, Apr. 2023.
- [44] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” 2019.
- [45] Yu. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs,” 2016.
- [46] W. Contributors, “Learning management system,” Sept. 2019.
- [47] O. Kiat Xin and D. Singh, “Development of learning analytics dashboard based on moodle learning management system,” *IJACSA) International Journal of Advanced Computer Science and Applications*, vol. 12, p. 2021, Jan. 2021.
- [48] K. Holstein, B. M. McLaren, and V. Aleven, “Intelligent tutors as teachers’ aides: Exploring teacher needs for real-time analytics in blended classrooms,” in *Proceedings of the Seventh International Learning Analytics & Knowledge Conference*, (Vancouver British Columbia Canada), pp. 257–266, ACM, Mar. 2017.
- [49] K. Holstein, B. M. McLaren, and V. Aleven, “Student Learning Benefits of a Mixed-Reality Teacher Awareness Tool in AI-Enhanced Classrooms,” in *Artificial Intelligence in Education* (C. Penstein Rosé, R. Martínez-Maldonado, H. U. Hoppe, R. Luckin, M. Mavrikis, K. Porayska-Pomsta, B. McLaren, and B. Du Boulay, eds.), vol. 10947, pp. 154–168, Cham: Springer International Publishing, 2018.
- [50] S. Lee-Cultura, K. Sharma, and M. N. Giannakos, “MultiModal teacher dashboards: Challenges and opportunities of enhancing teacher insights through a case study,” pp. 1–19, Jan. 2023.
- [51] L. P. Macfadyen and S. Dawson, “Mining LMS data to develop an “early warning system” for educators: A proof of concept,” *Computers & Education*, vol. 54, pp. 588–599, Feb. 2010.
- [52] D. Bañeres, M. E. Rodríguez, A. E. Guerrero-Roldán, and A. Karadeniz, “An early warning system to detect at-risk students in online higher education,” *Applied Sciences*, vol. 10, p. 4427, June 2020.
- [53] edInsight, “At-risk students early reporting system - edInsight,” Sept. 2020.

- [54] K. E. Arnold and M. D. Pistilli, “Course signals at Purdue: Using learning analytics to increase student success,” in *Proceedings of the 2nd International Conference on Learning Analytics and Knowledge*, LAK '12, (New York, NY, USA), pp. 267–270, Association for Computing Machinery, Apr. 2012.
- [55] S. M. Jayaprakash, E. W. Moody, E. J. M. Lauría, J. R. Regan, and J. D. Baron, “Early Alert of Academically At-Risk Students: An Open Source Analytics Initiative,” *Journal of Learning Analytics*, vol. 1, pp. 6–47, May 2014.
- [56] B. Efron and C. Morris, “Stein’s Paradox in Statistics,” *Scientific American*, vol. 236, pp. 119–127, May 1977.
- [57] C. N. Morris, “Parametric Empirical Bayes Inference: Theory and Applications,” *Journal of the American Statistical Association*, vol. 78, pp. 47–55, Mar. 1983.
- [58] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, “A Limited Memory Algorithm for Bound Constrained Optimization,” *SIAM Journal on Scientific Computing*, vol. 16, pp. 1190–1208, Sept. 1995.
- [59] R. Little and D. Rubin, *Statistical Analysis with Missing Data*. Wiley Series in Probability and Statistics, Wiley, 2014.
- [60] J. B. Schafer, D. Frankowski, J. Herlocker, and S. Sen, “Collaborative filtering recommender systems,” *The Adaptive Web*, vol. 4321, pp. 291–324, 2007.
- [61] F. L. da Silva, B. K. Slodkowski, K. K. A. da Silva, and S. C. Cazella, “A systematic literature review on educational recommender systems for teaching and learning: Research trends, limitations and opportunities,” *Education and Information Technologies*, vol. 28, pp. 3289–3328, Sept. 2022.
- [62] M. Deschênes, “Recommender systems to support learners’ Agency in a Learning Context: A systematic review,” *International Journal of Educational Technology in Higher Education*, vol. 17, Oct. 2020.
- [63] J. Li and Z. Ye, “Course recommendations in online education based on collaborative filtering recommendation algorithm,” *Complexity*, vol. 2020, p. e6619249, Dec. 2020.